

高斯过程回归与股价预测

随机过程期末大作业报告



小组成员 _____

专 业 _____

授课老师 _____ 杨恺、刘庆文

目录

一. 问题背景	3
1.1 实验背景	3
1.2 实验目的	3
1.3 实验方法	3
二. 理论分析	4
2.1 高斯分布与高斯过程回归	4
2.1.1 一元高斯分布	4
2.1.2 多元高斯分布	4
2.1.3 高斯过程	5
2.2 滑动窗口	6
2.3 涨跌预测概述	7
2.4 本次实验 GPR 部分理论分析概述	7
三. 实验步骤	8
3.1 实验环境	8
3.2 数据清洗与预处理	8
3.3 超参数的估计	10
3.4 核函数选择	10
四. 实验结果	12
4.1 ADBE 股票	13
4.2 ADSK 股票	15
4.3 MSFT 股票	18
4.4 ORCL 股票	18
4.5 SAP 股票	21
4.6 VRSN 股票	24
4.7 预测步长分析	26
4.8 涨跌预测分析	27
4.9 时间复杂度分析	34
五. 实验总结	36
参考文献	37

一.问题背景

1.1 实验背景

近年来，人工智能和机器学习领域的学者将更多的目光投向了“随机回归模型”，高斯过程回归（Gaussian Processes Regression, GPR）就是其中的典型代表。GPR 采用如下模型：

$$Y = g(X) + N$$

其中 $g(X)$ 是一个和 X 存在紧密联系的高斯过程的样本轨道片段。

而股票价格数据最能反映股票市场的波动变化,股价的预测也成为了投资者们一直关注的热门问题，也是 GPR 的一种典型应用，对于股票交易商来说是非常重要的工具。现有的预测模型在短期预测中显示出有效的结果，然而其准确性在长期预测中会下降。本实验将应用 GPR 来预测股市趋势，并选择了参考文献[5]和[6]中的股票来验证所提出的模型，旨在改进其预测性能和时间复杂度等，从而帮助投资者更好地进行长期投资或验证他们的投资决策。

1.2 实验目的

本实验旨在深入理解高斯过程回归（GPR）的底层原理，并通过 GPR 模型对 ADBE、ADSK、MSFT、ORCL、SAP、VRSN 股票（国外学生研究过的股票，但原始数据太久远，已经找不到）的历史收盘价进行预测，并从平均绝对百分比误差和置信水平两方面对比评估不同模型在不同大小历史数据集上的表现。

1.3 实验方法

具体来说，首先从 CSV 文件中读取股票的历史数据，包括日期和收盘价，确保数据按时间顺序排列并去除货币符号，将收盘价转换为浮点数。接着，使用 8:2 的比例将数据分为训练集和测试集，确保时间序列的顺序性，然后构建 GPR 模型，选择合适的核函数（如马特恩核和白噪声核），并对训练集进行拟合，生成预测值。实验探讨了使用不同大小的数据集、不同组合的核函数、GPR 结合滑动窗口/全局特性方法对模型预测效果的影响，通过计算平均绝对百分比误差和实际值落在预测上下限区间内的概率，评估模型的预测准确性，并绘制训练集、测试集的真实值和预测值，以及预测的不确定性区间，直观展示模型的表现。最后探讨可能的改进方向，如优化模型超参数、优化模型计算效率等。

1.4 成员分工

朱雨夏：GPR 结合滑动窗口、全局特性预测模型搭建及报告撰写

纪舜涵：GPR 预测股价涨跌模型搭建及滑动窗口优化及报告撰写

赵元锦：GPR 预测涨跌与全局股价对比分析报告撰写及 PPT 制作

二.理论分析

高斯过程回归（GPR）是一种非参数的回归方法，基于高斯过程理论。在机器学习中，高斯过程是一种强大的工具，用于建模连续型的函数关系，特别适用于小样本的情况。在进行预测时，高斯过程回归会计算给定输入下目标函数的后验分布，这个后验分布也是一个高斯分布，它的均值给出了预测值，方差则表示了预测的不确定性。由于高斯过程回归是基于贝叶斯推断的，因此它能够提供预测结果的不确定性估计，这对于许多应用场景非常有用，尤其是在决策制定中需要考虑风险的情况下。

高斯过程回归的优点包括：

- 能够灵活地处理非线性关系，适用于各种类型的函数逼近。
- 能够提供预测的不确定性，有利于进行决策制定。
- 不需要事先指定复杂的模型结构，减轻了参数调节的负担。

2.1 高斯分布与高斯过程回归

2.1.1 一元高斯分布

若随机变量 x 服从一个位置参数为 μ 、尺度参数为 σ 的概率分布，且其概率密度函数为：

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

则这个随机变量就称为正态随机变量，服从正态分布(也称为高斯分布)，记作 $x \sim N(\mu, \sigma^2)$ 。当 $\mu=0, \sigma=1$ 时成为标准正态分布：

$$f(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$

一元高斯分布有很多很好的性质，比如两个高斯分布的线性组合仍然是高斯分布，而多个一元高斯分布的联合分布则可以导出多元高斯分布。

2.1.2 多元高斯分布

若一个多维随机向量具有同一元高斯分布类似的概率规律时，称此随机向量遵从多元

高斯(multivariate Gaussian)分布。多元高斯分布可以从一元高斯分布导出，对于 D 维的随机变量 $\mathbf{x} = (x_1, \dots, x_D)^T$, 多元高斯分布形式为:

$$N(\mathbf{x} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\boldsymbol{\Sigma}|^{1/2}} \exp \left\{ -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right\}$$

式中的 $\boldsymbol{\mu}$ 是 D 维均值向量, $\boldsymbol{\Sigma}$ 是 $D \times D$ 的协方差矩阵 (covariance matrix), $|\boldsymbol{\Sigma}|$ 是 $\boldsymbol{\Sigma}$ 的行列式。多元正态分布也有很好的性质, 例如多元正态分布的边缘分布仍为正态分布, 它经任何线性变换得到的随机向量仍为多维正态分布, 它的线性组合为一元正态分布。给出多元高斯分布条件概率分布的定理:

设 $\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, 将 $\mathbf{y}, \boldsymbol{\mu}$ 和 $\boldsymbol{\Sigma}$ 分割成如下形式:

$$\mathbf{y} = \begin{bmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{bmatrix} \begin{matrix} n_1 \\ n_2 \end{matrix} \quad \boldsymbol{\mu} = \begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix} \begin{matrix} n_1 \\ n_2 \end{matrix} \\ \boldsymbol{\Sigma} = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \begin{matrix} n_1 \\ n_2 \end{matrix}$$

其中 $\mathbf{y}, \boldsymbol{\mu} \in \mathbb{R}^n$ 且 $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$ 。那么我们可以知道 $\mathbf{y}_1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \Sigma_{11})$, 进一步可以推出条件概率分布:

$$\mathbf{y}_2 | \mathbf{y}_1 \sim \mathcal{N}(\boldsymbol{\mu}_2 + \Sigma_{21} \Sigma_{11}^{-1} (\mathbf{y}_1 - \boldsymbol{\mu}_1), \Sigma_{22} - \Sigma_{21} \Sigma_{11}^{-1} \Sigma_{12})$$

2.1.3 高斯过程

高斯过程回归模型通常可以从权重空间视角和函数空间视角去解释, 这里从函数空间的视角推导。回归模型能用高斯过程表示, 比如贝叶斯线性回归模型:

$$f(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})^T \mathbf{w}, \text{ 其中 } \mathbf{w} \sim \mathcal{N}(\mathbf{0}, \Sigma_p)$$

$\Phi(\mathbf{x})$ 表示经过某种变换后的输入, 于是可以得到:

$$\begin{aligned} \mu(\mathbf{x}) &= \mathbb{E}[f(\mathbf{x})] = \boldsymbol{\phi}(\mathbf{x})^T \mathbb{E}[\mathbf{w}] = 0 \\ k(\mathbf{x}, \mathbf{x}') &= \mathbb{E}[f(\mathbf{x})f(\mathbf{x}')] = \boldsymbol{\phi}(\mathbf{x})^T \mathbb{E}[\mathbf{w}\mathbf{w}^T] \boldsymbol{\phi}(\mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^T \Sigma_p \boldsymbol{\phi}(\mathbf{x}') \end{aligned}$$

所以:

$$f(\mathbf{x}) \sim \mathcal{GP}(0, \boldsymbol{\phi}(\mathbf{x})^T \Sigma_p \boldsymbol{\phi}(\mathbf{x}'))$$

进一步, 考虑一个一般的带噪声的回归模型:

$$y = f(\mathbf{x}) + \varepsilon \quad f(\mathbf{x}) \sim \mathcal{GP}(\mu, k)$$

其中, $f(\mathbf{x})$ 是参数待定的高斯过程, 噪声满足 $\varepsilon \sim \mathcal{N}(0, \sigma_n^2)$ 。现在我们假设数据集 $\mathcal{D} = \{(\mathbf{x}_i, y_i) | i = 1, \dots, n\}$, 其中 $\mathbf{x}_i \in \mathbb{R}^d, y_i \in \mathbb{R}$ 。可以表示为矩阵形式 $\mathcal{D} = (\mathbf{X}, \mathbf{y})$, 其中 $\mathbf{X} \in \mathbb{R}^{n \times d}, \mathbf{y} \in \mathbb{R}^n$ 。当观测点满足回归模型时, 按照高斯过程的定义, 这些点的联合分布 $[f(\mathbf{x}_1), \dots, f(\mathbf{x}_n)]$ 需要满足一个多维高斯分布, 即:

$$[f(\mathbf{x}_1), f(\mathbf{x}_2), \dots, f(\mathbf{x}_n)]^T \sim \mathcal{N}(\boldsymbol{\mu}, K)$$

这里 $\boldsymbol{\mu} = [\mu(\mathbf{x}_1), \dots, \mu(\mathbf{x}_n)]^T$ 是均值向量, K 是 $n \times n$ 的矩阵, 其中第 (i, j) 个元素是 $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ 。

为了预测在新的点处的函数值 $f_* = f(Z)$, 其中 $Z = [z_1, \dots, z_m]^T$, 根据高斯分布的性质, 训练点和预测点的联合分布仍然为高斯分布, 即:

$$\begin{bmatrix} \mathbf{y} \\ f_* \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \boldsymbol{\mu}(X) \\ \boldsymbol{\mu}(Z) \end{bmatrix}, \begin{bmatrix} K(X, X) + \sigma_n^2 \mathbf{I} & K(Z, X)^T \\ K(Z, X) & K(Z, Z) \end{bmatrix} \right)$$

其中:

- $\mu(X) = \mu, \mu(Z) = [\mu(z_1), \dots, \mu(z_m)]^T, K(X, X) = K;$
- $K(Z, X)$ 是个 $m \times n$ 的矩阵, 其中第 (i, j) 个元素 $[K(Z, X)]_{ij} = k(z_i, x_j);$
- $K(Z, Z)$ 是个 $m \times m$ 的矩阵, 其中第 (i, j) 个元素 $[K(Z, Z)]_{ij} = k(z_i, z_j).$

最后利用高斯分布的条件分布性质, 可以得到关于预测值的条件概率分布:

$$p(f_* | X, y, Z) = \mathcal{N}(\hat{\mu}, \hat{\Sigma})$$

其中:

$$\begin{aligned}\hat{\mu} &= K(Z, X) (K(X, X) + \sigma_n^2 \mathbf{I})^{-1} (y - \mu(X)) + \mu(Z) \\ \hat{\Sigma} &= K(Z, Z) - K(Z, X) (K(X, X) + \sigma_n^2 \mathbf{I})^{-1} K(Z, X)^T\end{aligned}$$

如果将预测值的噪声考虑进来, 条件概率分布如下:

$$p(y_* | X, y, Z) = \mathcal{N}(\hat{\mu}, \hat{\Sigma} + \sigma_n^2 \mathbf{I})$$

在实际应用中通常将令均值函数 $\mu(x) = 0$ (本次实验也是这么做的), 则预测的均值函数和预测的协方差函数的结果可以更为简单的表示为:

$$\begin{aligned}\hat{\mu} &= K(Z, X)(K(X, X) + \sigma_n^2 \mathbf{I})^{-1} y \\ \hat{\Sigma} &= K(Z, Z) - K(Z, X)(K(X, X) + \sigma_n^2 \mathbf{I})^{-1} K(Z, X)^T\end{aligned}$$

至此, 完成了从函数空间视角出发的高斯过程回归模型的推导。

如果换一种视角看均值:

$$\hat{\mu}_i = \sum_{j=1}^n \alpha_j k(x_j, z_i)$$

最终结果表示, 预测值 $\hat{\mu}$ 可以看做是预测点和观测点之间核函数的线性组合, 这个方程表明高斯过程回归等价于使用核函数 $k(x, x')$ 将输入投影到特征空间的线性回归模型。为了得到预测点的值, 每个输出 $\hat{\mu}_i$ 被加权为其相关的输入 x_j 与被预测点 z_i 的相似度, 这是由核函数诱导的相似度度量。这个加权和表明, 高斯过程回归模型取决于选定的核函数和到目前为止观察到的数据, 而不需要指定具体的模型函数形式, 这就是为什么高斯过程回归是一种非参数方法。

用高斯过程对回归问题进行建模有很多优势。首先, GP 利用有限的训练数据点计算核函数, 建模复杂度与 x 的维数无关, 使得我们能够处理更高维的输入数据, 理论上无限维。其次, 高斯过程是一种非参数的方法, 对回归问题进行建模时, 不需要指定模型的形式, 这意味着高斯过程可以对任意形式的函数进行建模拟合。

2.2 滑动窗口

本次实验中, 滑动窗口法被用来构建时间序列预测模型。该方法的核心思想是通过创建一系列固定长度的子序列(即窗口), 每个窗口包含连续的过去 `window_size=30` 个时间点的数据, 用以预测紧接着这个窗口之后的一段未来时间的数据值(设定步长 `pred_steps`)。

具体来说, 整个原始时间序列数据首先被转换为特征矩阵 X 和目标向量 y , 其中 X 由每个窗口内的历史数据构成, 而 y 则是紧接在每个窗口之后的 `pred_steps` 天期间的真实值。

然后，这些数据按照时间顺序被划分为训练集和测试集（8: 2），确保了训练集包含较早的数据，而测试集则包含了最近的数据，以此来模拟实际的预测场景。

在训练阶段，模型学习到了如何根据过去的 `window_size` 个时间点的数据来预测下一时间段的值。而在测试阶段，模型采用滚动预测的方式：从测试集的第一个窗口开始，基于之前的历史数据进行预测，随后将实际观测到的新数据加入到已知数据中，用于下一次预测的输入，这一过程持续到遍历完所有测试数据。这种方法不仅捕捉了时间序列中的动态变化，还模拟了实时预测的过程，使得模型能够更好地适应未来的数据变化。通过这种方式，我们可以评估模型对未来真实数据的预测能力，并计算预测误差和其他性能指标，从而判断模型的有效性。

2.3 涨跌预测概述

本次实验中，我们应用高斯分类与高斯回归两种方式对股票的连续单日涨跌进行预测，目的是想通过涨跌预测的准确度，结合股票实际价格的走向，来分析已构建模型对于实际预测任务的预测效果评估。

对于高斯分类，在处理数据时，我们计算了技术指标（如移动平均、RSI 等）作为特征，更侧重于使用技术指标来辅助模型理解市场动态，在实现过程中，还包括了数据的标准化处理、模型训练时的异常处理、以及通过混淆矩阵、ROC 曲线等方法对模型性能的全面评估。而对于高斯回归，直接使用过去一段时间内的股票价格作为特征，通过滑动窗口技术构建时间序列数据，这种方法依赖于模型从价格序列本身学习价格变动的模式。在评估模型时，高斯回归更侧重于预测价格的准确性，并通过准确率和分类报告来衡量模型性能。

高斯分类预测的原理在于确定样本属于哪个预先定义类别。在股票涨跌预测中，分类模型会分析历史数据并学习特征与涨跌结果之间的关联，从而对新的数据点进行类别预测。这种方法适用于目标变量是离散的情况。

而高斯回归预测的原理则是预测一个连续的数值，它试图找到输入特征和连续输出变量之间的数学关系。在股票价格预测中，回归模型会根据历史价格数据学习价格随时间变化的趋势，以便对将来的价格进行预测。这种方法适用于目标变量是连续的情况。

虽然两种方法在特征处理、模型训练和性能评估方面有所不同，但都旨在从历史数据中学习并对未来的股票市场行为做出准确的预测。

2.4 本次实验 GPR 部分理论分析概述

本次实验选用各支股票的收盘价格和收盘时间。设 x 为收盘时间， f 为收盘时间对应的收盘价格，先假设无噪声，且对股票收盘价格进行去均值的预处理。则有：

$$f \sim GP(0, K)$$

其中 $K = K(x, x)$ 为协方差矩阵，选择不同的核函数来拟合会导致 K 不同，如常见的 SE 核：

$$k_{ij} = \sigma_f^2 \exp\left(-\frac{(x_i - x_j)^T (x_i - x_j)}{2\rho^2}\right)$$

现用股票的前一段收盘时间和收盘价格作为训练集 x, f ；后一段收盘时间作为测试集 x^* ，预测测试集 x^* 对应的收盘价格 f^* ，则有：

$$\begin{bmatrix} f \\ f^* \end{bmatrix} \sim N\left(0, \begin{bmatrix} K(x, x) & K(x, x^*) \\ K(x^*, x) & K(x^*, x^*) \end{bmatrix}\right)$$

由贝叶斯后验公式：

$$f^* \sim N(\mu^*, \sigma^*)$$

其中, $\mu^* = K(x, x^*)^T K^{-1} f, \sigma^* = K(x^*, x^*) - K(x, x^*)^T K^{-1} K(x, x^*)$

有噪音的情况下, 即:

$$y = f(x) + n, n \sim N(0, \sigma_n^2)$$

$$\mu^* = K(x, x^*)^T (K + I\sigma_n^2)^{-1} f, \sigma^* = K(x^*, x^*) - K(x, x^*)^T (K + I\sigma_n^2)^{-1} K(x, x^*)$$

即训练模型的超参数为 σ_f, ρ, σ_n

三.实验步骤

3.1 实验环境

本实验使用 Python3.11 环境，依赖于多个常用的科学计算和机器学习库，包括 pandas、numpy、scikit-learn、matplotlib 等

3.2 数据清洗与预处理

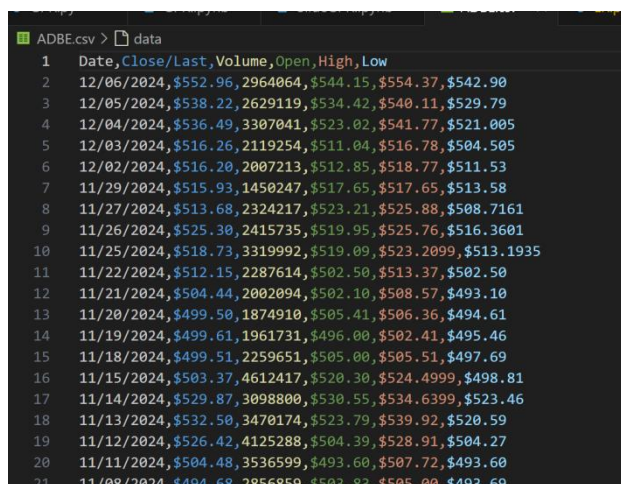
实验选用股票为参考文献中国外学生研究过的几种股票和 NVIDIA 股票最近 10 年交易日的数据（每只股票 2516 个条记录），包括：ADBE,ADSK,MSFT,OCRL,SAP,VRSN，共 6 只。

◆ 数据读取

使用 pandas 库读取包含股票历史数据的 CSV 文件。该文件包含日期和收盘价等信息。为了确保数据按时间顺序排列，首先检查数据是否已经按日期升序排列，如果不是，则对其进行排序。此外，去除收盘价列中的货币符号（如美元符号），并将其转换为浮点数，以便后续的数值计算。

```
# 读取数据
data1 = pd.read_csv('NVIDIA.csv', parse_dates=['Date'], index_col='Date')
# 检查数据是否已经是按日期升序排列
if not data1.index.is_monotonic_increasing:
    # 如果不是升序排列，则进行排序
    data1 = data1.sort_index(ascending=True)
# 去除 'Close/Last' 列中的货币符号并转换为浮点数
data1['Close/Last'] = data1['Close/Last'].str.replace(r'[\d.]', '', regex=True).astype(float)
stock1 = data1['Close/Last'].values
```

原始数据：



	Date	Close/Last	Volume	Open	High	Low
1	12/06/2024	\$552.96	2964064	\$544.15	\$554.37	\$542.90
2	12/05/2024	\$538.22	2629119	\$534.42	\$540.11	\$529.79
3	12/04/2024	\$536.49	3307041	\$523.02	\$541.77	\$521.005
4	12/03/2024	\$516.26	2119254	\$511.04	\$516.78	\$504.505
5	12/02/2024	\$516.20	2007213	\$512.85	\$518.77	\$511.53
6	11/29/2024	\$515.93	1450247	\$517.65	\$517.65	\$513.58
7	11/27/2024	\$513.68	2324217	\$523.21	\$525.88	\$508.7161
8	11/26/2024	\$525.30	2415735	\$519.95	\$525.76	\$516.3601
9	11/25/2024	\$518.73	3319992	\$519.09	\$523.2099	\$513.1935
10	11/22/2024	\$512.15	2287614	\$502.50	\$513.37	\$502.50
11	11/21/2024	\$504.44	2002094	\$502.10	\$508.57	\$493.10
12	11/20/2024	\$499.50	1874910	\$505.41	\$506.36	\$494.61
13	11/19/2024	\$499.61	1961731	\$496.00	\$502.41	\$495.46
14	11/18/2024	\$499.51	2259651	\$505.00	\$505.51	\$497.69
15	11/15/2024	\$503.37	4612417	\$520.30	\$524.4999	\$498.81
16	11/14/2024	\$529.87	3098800	\$530.55	\$534.6399	\$523.46
17	11/13/2024	\$532.50	3470174	\$523.79	\$539.92	\$520.59
18	11/12/2024	\$526.42	4125288	\$504.39	\$528.91	\$504.27
19	11/11/2024	\$504.48	3536599	\$493.60	\$507.72	\$493.60
20	11/08/2024	\$494.68	2856859	\$503.83	\$505.00	\$493.69
21						

处理后：

	Close/Last	Volume	Open	High	Low
Date					
2014-12-09	71.565	2866736	\$71.34	\$71.66	\$70.68
2014-12-10	69.850	2573489	\$71.56	\$71.97	\$69.74
2014-12-11	69.740	5081614	\$70.10	\$71.14	\$69.66
2014-12-12	76.020	12603700	\$74.17	\$77.56	\$73.71
2014-12-15	73.040	9652775	\$75.66	\$76.13	\$72.84
...	...	...	...	...	...
2024-12-02	516.200	2007213	\$512.85	\$518.77	\$511.53
2024-12-03	516.260	2119254	\$511.04	\$516.78	\$504.505
2024-12-04	536.490	3307041	\$523.02	\$541.77	\$521.005
2024-12-05	538.220	2629119	\$534.42	\$540.11	\$529.79
2024-12-06	552.960	2964064	\$544.15	\$554.37	\$542.90

2516 rows x 5 columns

◆ 创建滑动窗口数据集，划分训练集测试集

滑动窗口用 30 期预测 `pred_steps` 期：定义 `window_size` 参数，设为 30，表示每个窗口包含过去 30 个时间点的数据，接着定义 `create_sliding_window` 函数，该函数遍历整个时间序列数据 `data`，对于每一个可能的起始位置，提取出长度为 `window_size` 的连续子序列作为特征向量 `X`，并将其紧接着的 `pred_steps` 时间段的数据值作为目标值 `y`。原始的时间序列到这里被转换成了多个 $(\text{len}(\text{data}) - \text{window_size})$ 对特征-目标值组合，其中每个特征向量代表了一段历史数据，而对应的目标值则是这段历史之后的下一个 `pred_steps` 时间段的实际观测值。

再将构造好的特征矩阵 `X` 和目标向量 `y` 按照 8:2 划分为训练集和测试集，且保持时间顺序（不打乱数据），以确保训练集由较早的数据组成，而测试集则由最近的数据构成，以此来模拟实际的预测环境。

```
# 创建滑动窗口数据集
def create_sliding_window(data, window_size, pred_steps=1):
    X, y = [], []
    for i in range(len(data) - window_size - pred_steps + 1):
        # 输入窗口
        window = data[i:(i + window_size)]
        X.append(window)

        # 目标值可以是未来多步预测
        target = data[i + window_size:i + window_size + pred_steps]
        y.append(target)

    return np.array(X), np.array(y)

# 多步预测
pred_steps = 5

# 使用滑动窗口创建训练集和测试集
X, y = create_sliding_window(stock1, window_size, pred_steps)
trainX, testX, trainY, testYreal = train_test_split(X, y, test_size=0.2, shuffle=False)
```

◆ 不使用滑动窗口，直接划分训练集测试集

为方便训练和测试集划分，首先创建一个时间索引，表示每个观测值的时间顺

序。时间索引从 1 开始，依次递增，直到最后一个观测值。

按照 8:2 的比例将数据划分为训练集和测试集。为了保持时间序列的顺序性，禁用随机打乱（shuffle=False），确保训练集在前，测试集在后。这样可以模拟实际的预测场景，即使用过去的的数据来预测未来的股价。

```
# 创建时间索引
time_index = np.arange(1, len(stock1) + 1).reshape(-1, 1)
# 按8:2比例划分训练集和测试集
trainX, testX, trainY, testYreal = train_test_split(time_index, stock1, test_size=0.15, shuffle=False)
```

◆ 数据标准化（去均值化）

为减少计算量，假设进行 GPR 的数据的均值为 0。但实际上因为股票价格不能为负数，均值不可能为 0。对此，预先对训练集数据进行去均值化处理，即：

$$\text{TrainY} = \text{TrainY} - \text{avg}(\text{TrainY})$$

但要注意，预测出未来的股票价格后需再将其加上 avg(TrainY)还原

```
# 标准化数据
scaler_X = StandardScaler()
scaler_y = StandardScaler()

trainX_scaled = scaler_X.fit_transform(trainX)
trainY_scaled = scaler_y.fit_transform(trainY)

testX_scaled = scaler_X.transform(testX)
```

3.3 超参数的估计

在本次实验中，在从指定个随机起点开始寻找核函数超参数的最佳值时，使用了 L-BFGS-B 优化算法进行模型超参数的估计，这是一种拟牛顿法，通过构建 Hessian 矩阵的近似来避免直接计算和求逆 Hessian 矩阵，从而减少每次迭代的计算成本，提高计算效率。

3.4 核函数选择

实验选择了多组核函数，对其预测效果进行对比分析，具体说明如下：

3.4.1 Matérn 函数+ WhiteKernel 函数

```
# GPR模型:Matern 和 WhiteKernel 组合
kernel = ConstantKernel() * Matern(length_scale=1.0, nu=1.5) + WhiteKernel(noise_level=1)
```

◆ Matérn 函数

$$k_{\text{Matern}}(x, x') = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu} \frac{|x-x'|}{l} \right)^\nu K_\nu \left(\sqrt{2\nu} \frac{|x-x'|}{l} \right)$$

其中， σ^2 是信号方差， l 是长度尺度参数， ν 控制平滑度， K 是修正的 Bessel 函数。此处选择 Matern3/2，因为其效果好于 Matern5/2

◆ WhiteKernel 函数

$$k_{\text{White}}(x, x') = \sigma_n^2 \delta(x - x')$$

其中， σ_n^2 是噪声方差， δ 是 Dirac delta 函数，表示只有当 $x=x'$ ，核值为 σ_n^2 ，否则为 0。

◆ 说明：

Matérn 核提供了从绝对不连续到无限可微的函数空间之间的灵活性。通过调

整 ν 参数，可以控制输出函数的平滑度：较大的 ν 值对应更平滑的函数。

WhiteKernel 用于建模观测噪声，确保模型能够处理含有噪声的数据。它只对同一位置的点有贡献，因此不会影响不同位置之间的相关性。

- ◆ 该组合的适用情况：适用于具有中等平滑度的时间序列或空间数据，特别是当数据中存在显著的噪声时。另外，这 **Matern** 核函数不带周期项，对初值不太敏感，有利于模型预测。

3.4.2 Rational Quadratic * Periodic + WhiteKernel 函数

```
# GPR模型: Rational Quadratic * Periodic + WhiteKernel
kernel = ConstantKernel() * RationalQuadratic(length_scale=1.0, alpha=1.0) * ExpSineSquared(length_scale=1.0, periodicity=30) + WhiteKernel(noise_level=1)
```

◆ Rational Quadratic 函数

$$k_{\text{RQ}}(x, x') = \sigma^2 \left(1 + \frac{(x-x')^2}{2\alpha l^2} \right)^{-\alpha}$$

◆ Periodic 函数

$$k_{\text{Per}}(x, x') = \sigma^2 \exp \left(-\frac{2 \sin^2(\pi |x-x'|/p)}{l^2} \right)$$

其中， p 是周期参数， l 是长度尺度。

◆ 说明：

Rational Quadratic 核可以看作是多个不同长度尺度的 **RBF** 核的加权平均，因此它可以捕捉多尺度的相关性。参数 α 控制着不同长度尺度的重要性：较小的 α 表示更多权重放在较短长度尺度上，而较大的 α 则倾向于长度尺度。

Periodic 核用于捕捉数据中的周期性成分，例如季节效应或日周期变化。

WhiteKernel 用于建模观测噪声，确保模型能够处理含有噪声的数据。它只对同一位置的点有贡献，因此不会影响不同位置之间的相关性。

Rational Quadratic 和 **Periodic** 的乘积结合多了尺度相关性和周期性，使得模型能够捕捉到既有短期快速变化又有长期趋势的时间序列特征。

- ◆ 该组合的适用情况：适用于既包含周期性又有多尺度变化的时间序列，如经济指标或生物节律数据。由于股价时间序列可能具有周期性特征，还可能包含多个不同的时间尺度变化以及观测噪声，所以核函数的这种组合可以帮助模型更好地理解复杂的时间依赖关系，提供更加精确的预测。

四.实验结果

为评估模型性能，以下计算了平均绝对百分比误差（MAPE），反映了预测值与真实值之间的平均偏差程度；还计算了实际值落在预测上下限区间内的概率，衡量了模型预测区间的置信度。两个指标共同提供关于模型准确性及其预测不确定性的量化评价。

另外，以折线图直观展示预测结果。蓝色线条表示全部的历史收盘价，绿色线条代表测试集的真实收盘价，紫色线条展示了模型对测试集的预测结果。此外，图中还用浅色填充区域标识了 95%置信水平下的预测区间，即上下限范围。由折线图观察到模型预测的趋势以及它与实际市场变动的吻合程度，同时也能够看到预测的不确定性范围。

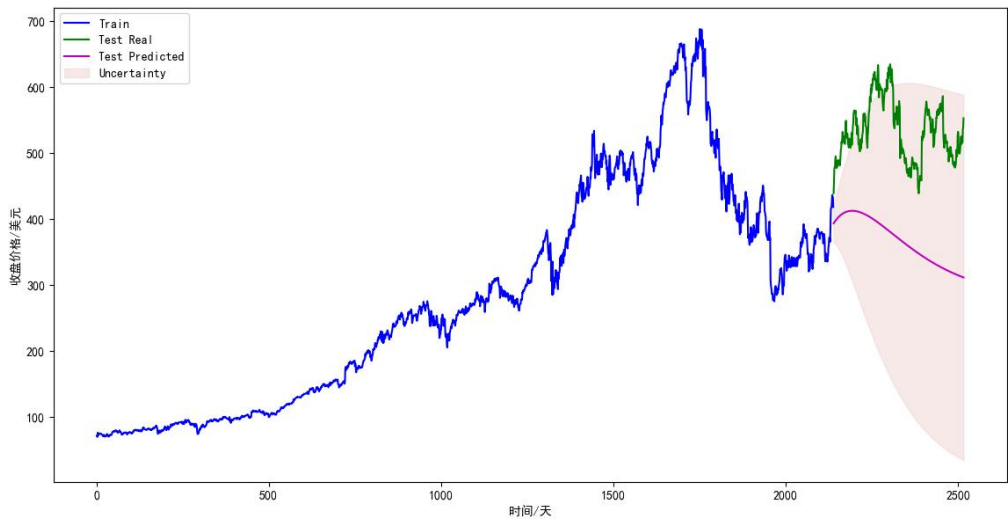
此处使用的 `pred_steps` 是 1，即滑动窗口只对后面一天进行预测。

另外，只选取了 4.1 4.2 4.6 三只股票做了相应方法结果对比分析，其余结果的产生原因类似。

4.1 ADBE 股票

4.1.1 Matérn 函数+ WhiteKernel

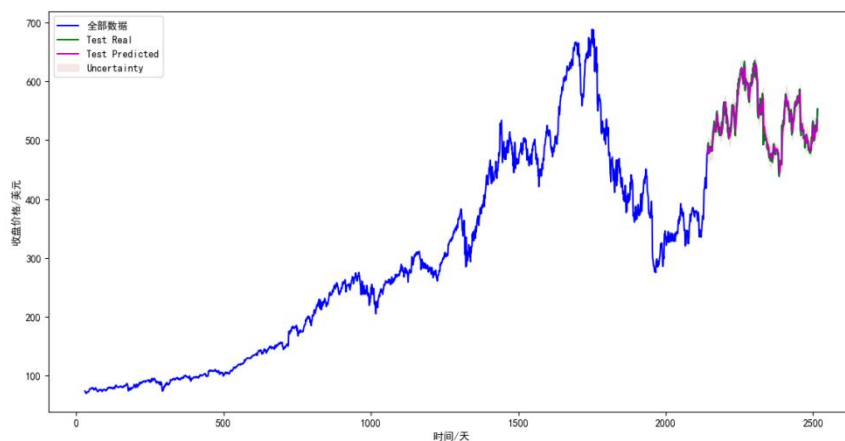
不使用滑动窗口（全局特征）+10 年数据：



平均绝对百分比误差为：30.71%

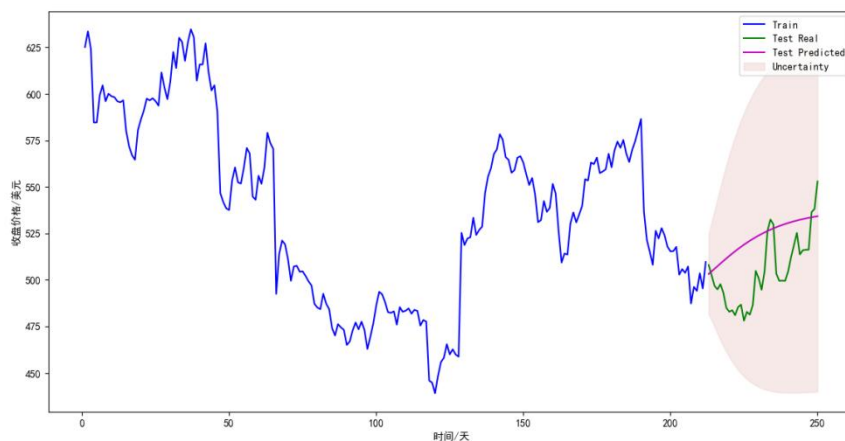
实际值在预测上下限区间的概率为：67.72%

使用滑动窗口+10 年数据：



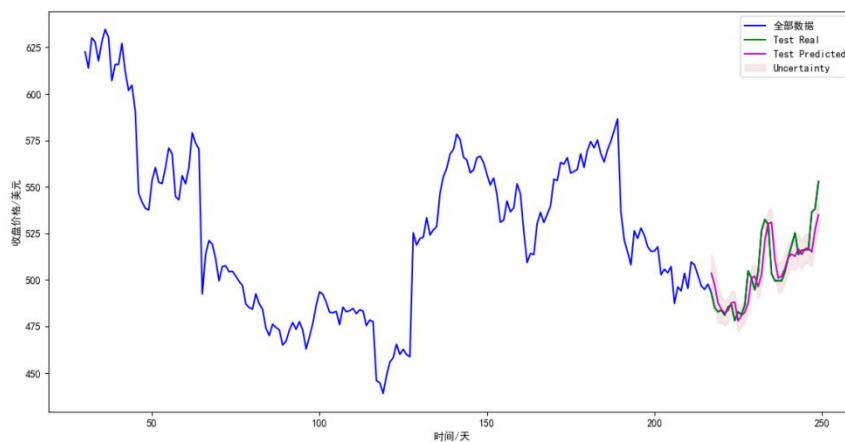
平均绝对百分比误差为：1.58%
实际值在预测上下限区间的概率为：79.62%

不使用滑动窗口+最近 1 年数据：



平均绝对百分比误差为：4.07%
实际值在预测上下限区间的概率为：100.00%

滑动窗口+最近 1 年数据：



平均绝对百分比误差为：1.42%
实际值在预测上下限区间的概率为：60.61%

4.1.2 数据集大小对比分析

根据结果，使用 Matérn 函数和 WhiteKernel 组合的 GPR 模型在不使用滑动窗口的情况下，MAPE 高达 30.71%，实际值落在预测上下限区间内的概率为 67.72%。表明模型未能有效捕捉时间序列中的短期波动和局部模式，导致预测值与真实值之间的差异较大。尽管不确定性估计相对合理，但预测偏差仍然显著，且全局特征处理方法可能丢失了重要的局部时间依赖性，使得模型难以适应复杂的动态变化。

相比之下，使用滑动窗口的方法显著提升了预测的准确性。MAPE 大幅降低至 1.58%，实际值在预测上下限区间的概率提高到 79.62%。滑动窗口方法通过利用固定长度的历史数据来预测未来的一个时间点，有效地捕捉了时间序列中的短期趋势、季节性和其他时间依赖性。这种方法不仅提高了预测精度，还提供了更为合理的不确定性估计，既不过于保守也不过于激进。Matérn 核函数与 WhiteKernel 的结合进一步增强了模型对噪声的鲁棒性和对复杂动态变化的适应能力。

可见，滑动窗口对于具有较强时间依赖性和短期波动的股票价格预测任务是更好的选择，能够在保持较高预测精度的同时，提供更可靠的置信区间估计

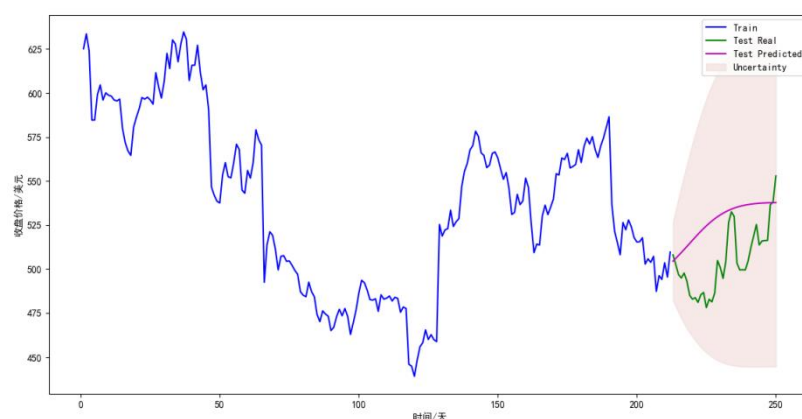
但另做实验证明，如果使用少量数据（近一年），不使用滑动窗口的 GPR 预测准确率和置信度都会上升，使用滑动窗口的准确率略有上升而置信度下降

可能是因为不使用滑动窗口的方法将整个时间序列作为输入，能够更好地利用有限的的数据，捕捉到整体的趋势和模式。尽管这种方法可能会丢失一些局部的时间依赖性，但在小数据集上，全局特征往往更为稳定和可靠。因此，不使用滑动窗口的 GPR 模型在这种情况下表现更好，MAPE 降低至 4.07%，并且 100% 的实际值落在预测上下限区间内，表明模型对不确定性的估计非常保守。另外，在小数据集上，使用全局特征的 GPR 模型相对简单，避免了过多的参数和复杂的结构，减少了过拟合的风险。这种简化模型能够在有限的的数据中找到较为稳定的模式，因此预测精度较高。

以下均以不使用滑动窗口 GPR+1 年数据、滑动窗口 GPR+10 年数据，并且 pred_steps=1 为例

4.1.3 Rational Quadratic * Periodic + WhiteKernel 函数

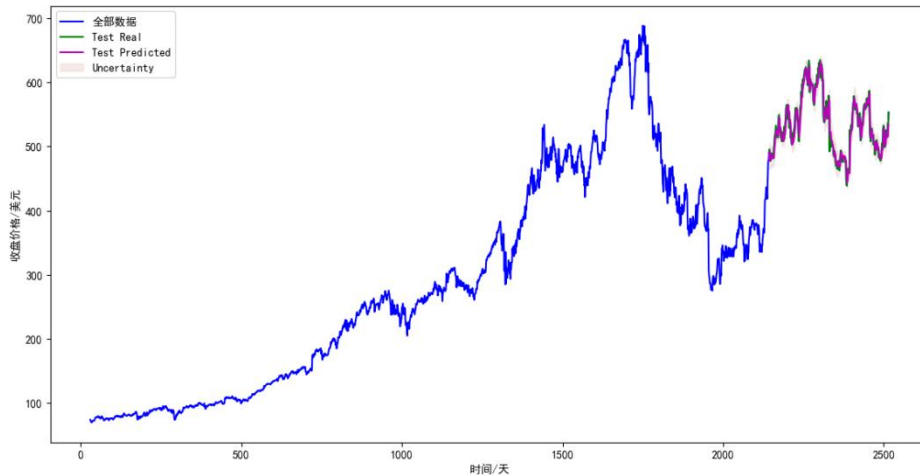
全局特征 GPR（1 年数据）：



平均绝对百分比误差为：4.87%

实际值在预测上下限区间的概率为：100.00%

使用滑动窗口 GPR（10 年数据）：



平均绝对百分比误差为：1.55%

实际值在预测上下限区间的概率为：78.55%

4.1.4 不同核函数组合结果对比分析

Matérn + WhiteKernel:

10 年数据：滑动窗口方法显著优于全局特征方法，能够更好地捕捉短期动态和局部模式，提供更高的预测精度和合理的置信度。

1 年数据：全局特征方法在小数据集上表现更好，能够捕捉整体趋势，避免过拟合并提供更高的置信度。滑动窗口方法虽然 MAPE 略有下降，但置信度显著下降，表明其在小数据集上的稳定性较差。

Rational Quadratic * Periodic + WhiteKernel:

1 年数据：全局特征方法仍然表现良好，能够捕捉整体趋势，提供较高的置信度。

Rational Quadratic * Periodic 核函数组合适用于具有多种时间尺度特征的数据，但 MAPE 略高于 Matérn + WhiteKernel 组合。

10 年数据：滑动窗口方法同样表现出色，能够捕捉多尺度的相关性和周期性，提供较高的预测精度和合理的置信度。

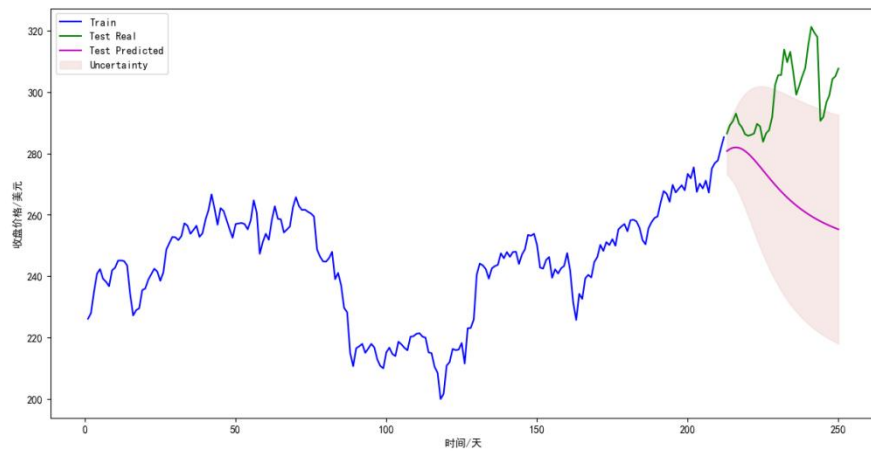
总而言之，对于大数据集，滑动窗口方法在两种核函数组合下都表现出了显著的优势，能够更好地捕捉时间序列中的短期动态和局部模式，提供更高的预测精度和合理的置信度。特别是 Rational Quadratic * Periodic + WhiteKernel 组合在处理具有复杂动态变化的时间序列时表现出色。

对于小数据集，全局特征方法（不使用滑动窗口）表现更好，能够捕捉整体趋势，避免过拟合并提供更高的置信度。Matérn + WhiteKernel 组合在小数据集上略优于 Rational Quadratic * Periodic + WhiteKernel 组合，可能是因为前者对噪声的鲁棒性更强，更适合处理有限的的数据。

4.2 ADSK 股票

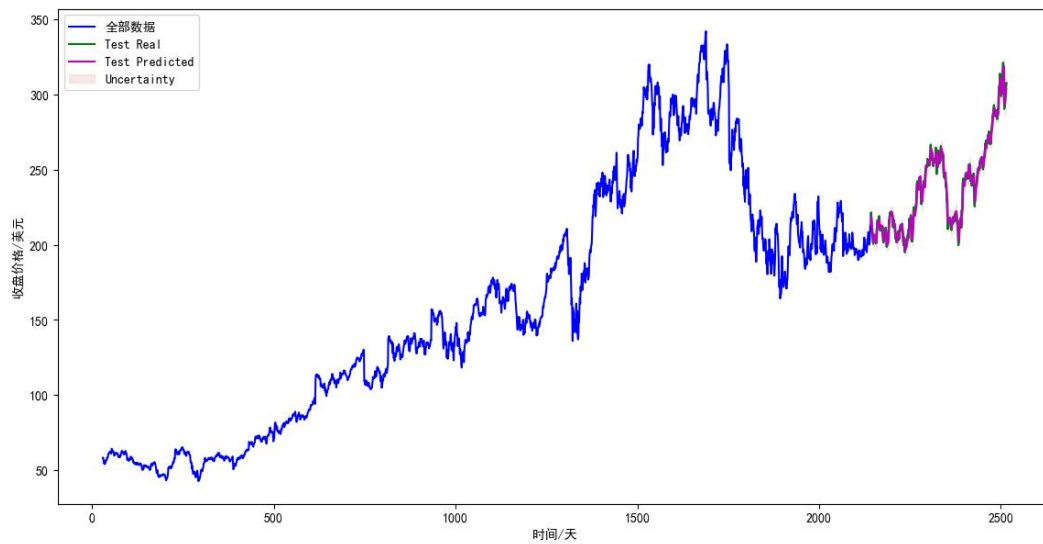
4.2.1 Matérn 函数+ WhiteKernel

全局特征 GPR（1 年数据）：



平均绝对百分比误差为：9.96%
实际值在预测上下限区间的概率为：47.37%

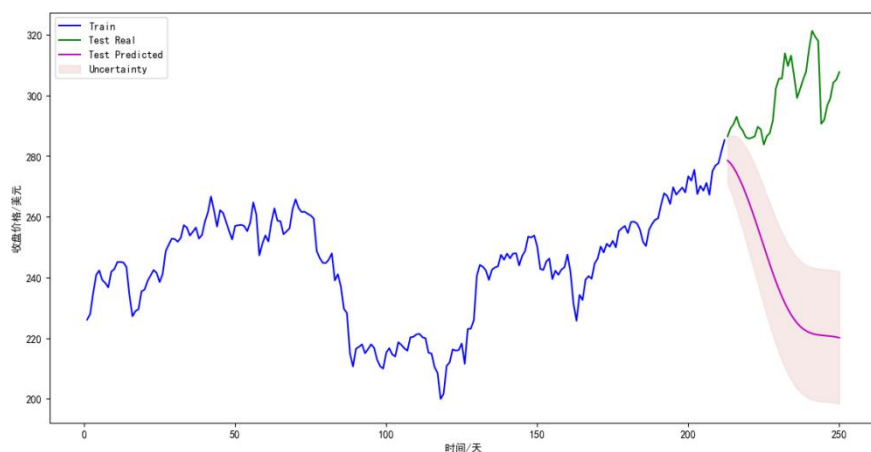
使用滑动窗口 GPR（10 年数据）：



平均绝对百分比误差为：1.43%
实际值在预测上下限区间的概率为：74.26%

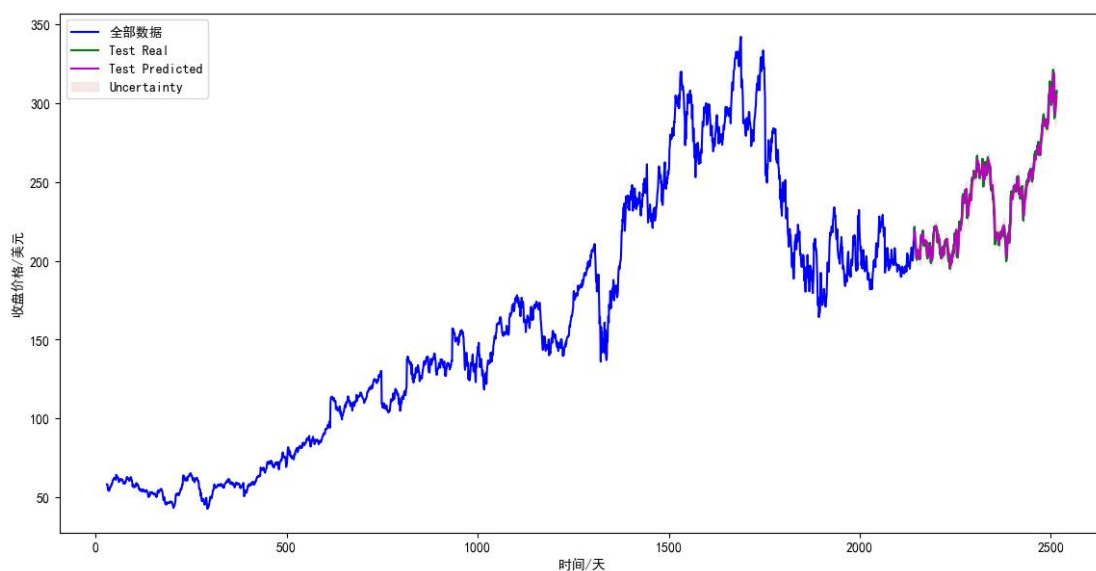
4.2.2 Rational Quadratic * Periodic + WhiteKernel 函数

全局特征 GPR（1 年数据）：



平均绝对百分比误差为：19.15%
实际值在预测上下限区间的概率为：0.00%

使用滑动窗口 GPR（10 年数据）：



平均绝对百分比误差为：1.42%
实际值在预测上下限区间的概率为：71.85%

4.2.3 对比分析

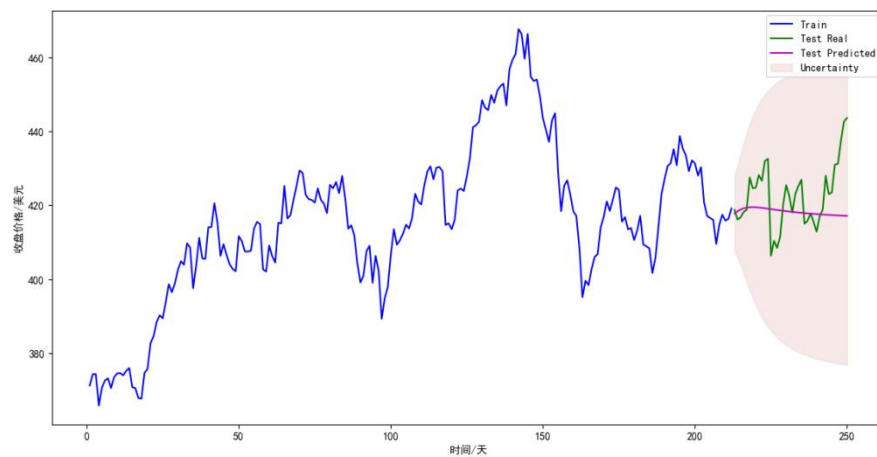
由上述结果，滑动窗口方法在两种核函数组合的大数据集上表现优异，能够有效捕捉短期动态和局部模式，提供更高的预测精度和合理的置信度。而对于小数据集，全局特征方法虽然在某些情况下（如 Matérn + WhiteKernel 组合）表现出一定的稳定性，但其预测精度和置信度仍然有限。

另外，Rational Quadratic * Periodic + WhiteKernel 组合在小数据集上的表现非常差，几乎无法提供有用的预测结果。这主要是因为全局特征方法更容易受到噪声和异常值的影响。

4.3 MSFT 股票

4.3.1 Matérn 函数+ WhiteKernel

全局特征 GPR（1 年数据）：

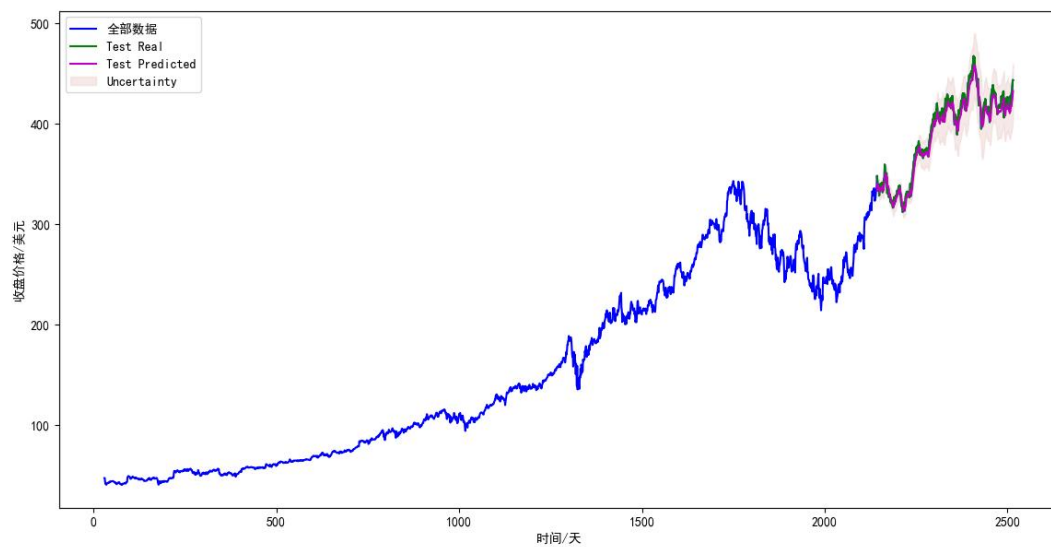


平均绝对百分比误差为：1.70%

实际值在预测上下限区间的概率为：100.00%

预测效果很好

使用滑动窗口 GPR（10 年数据）：

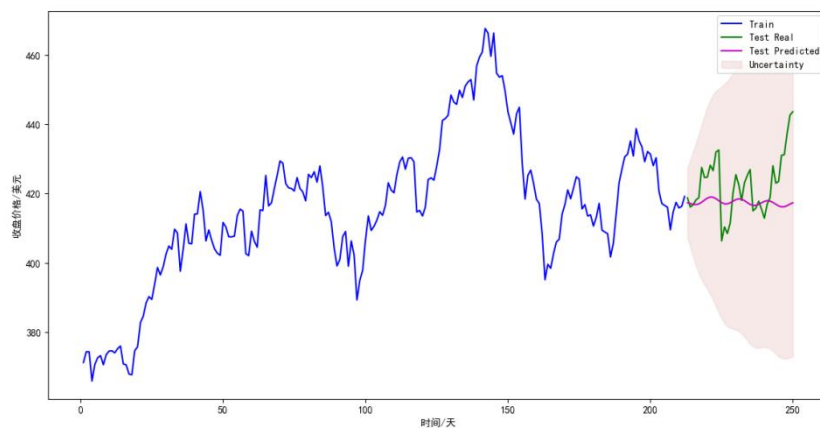


平均绝对百分比误差为：1.44%

实际值在预测上下限区间的概率为：95.71%

4.3.2 Rational Quadratic * Periodic + WhiteKernel 函数

全局特征 GPR（1 年数据）：

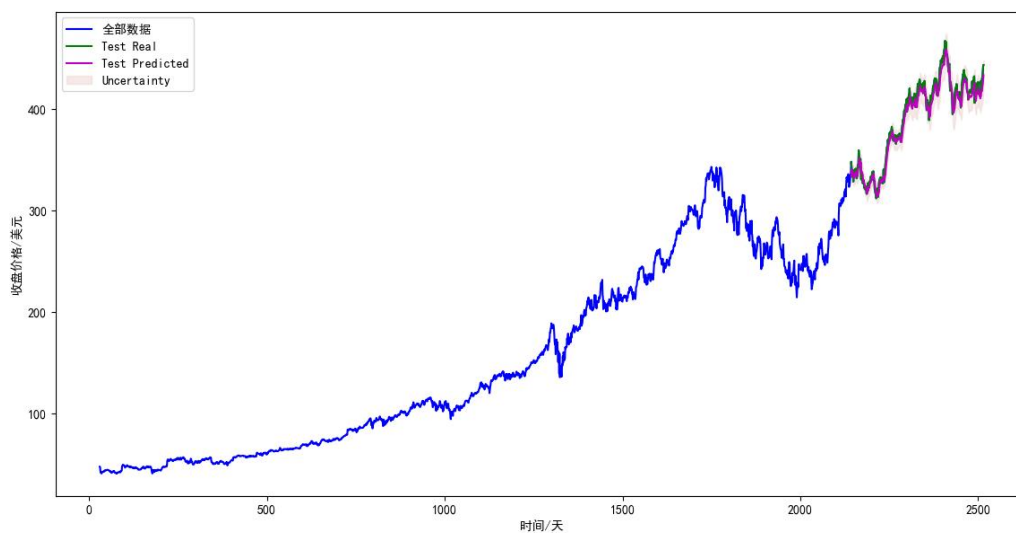


平均绝对百分比误差为：1.72%

实际值在预测上下限区间的概率为：100.00%

预测效果也很好

使用滑动窗口 GPR（10 年数据）：



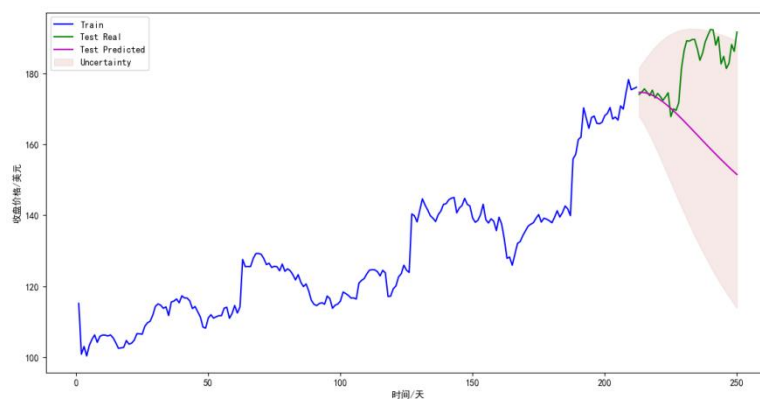
平均绝对百分比误差为：1.38%

实际值在预测上下限区间的概率为：90.08%

4.4 ORCL 股票

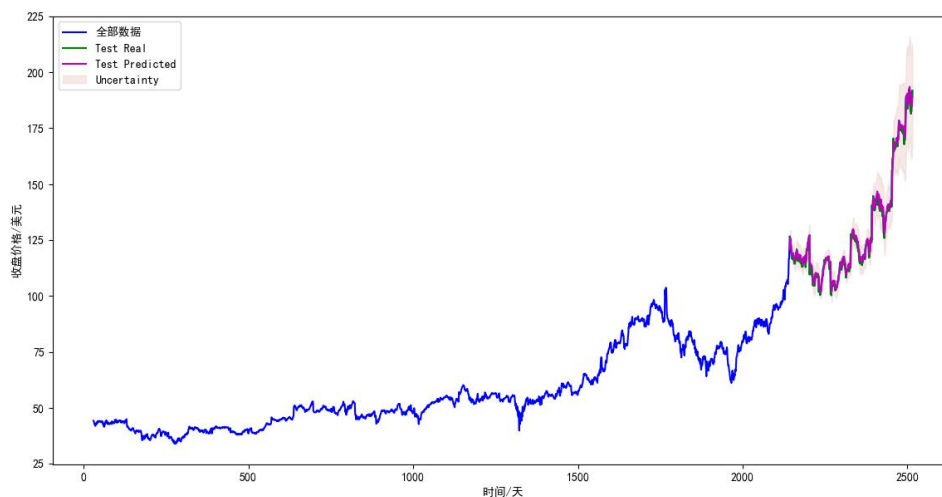
4.4.1 Matérn 函数+ WhiteKernel

全局特征 GPR（1 年数据）：



平均绝对百分比误差为：8.99%
实际值在预测上下限区间的概率为：92.11%

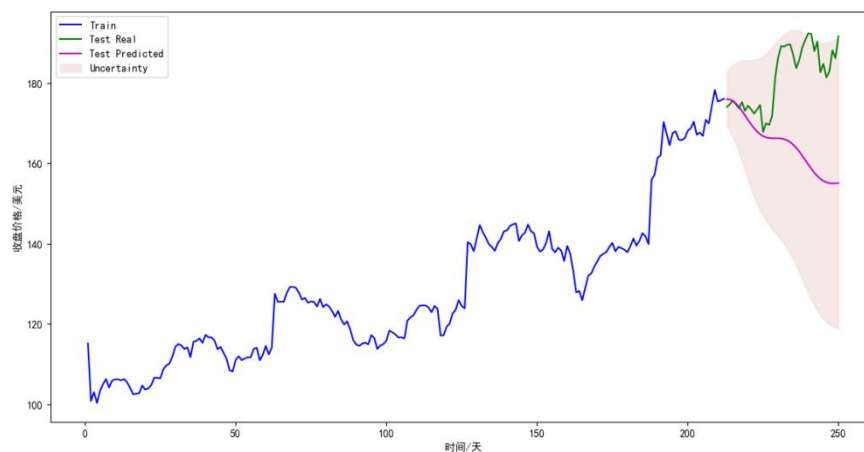
使用滑动窗口 GPR（10 年数据）：



平均绝对百分比误差为：1.45%
实际值在预测上下限区间的概率为：96.25%

4.4.2 Rational Quadratic * Periodic + WhiteKernel 函数

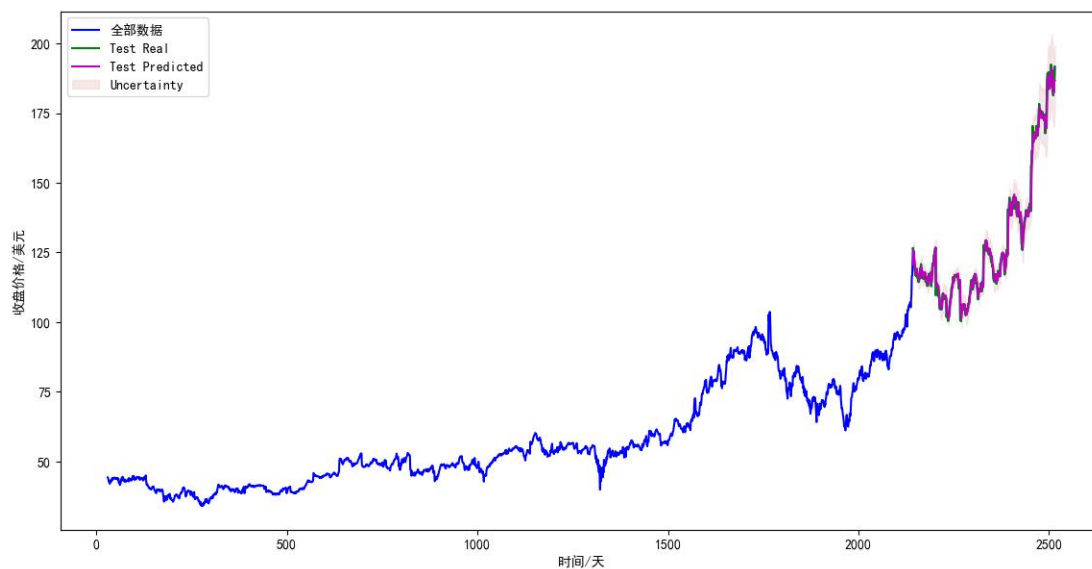
全局特征 GPR（1 年数据）：



平均绝对百分比误差为：8.97%

实际值在预测上下限区间的概率为：92.11%

使用滑动窗口 GPR（10 年数据）：



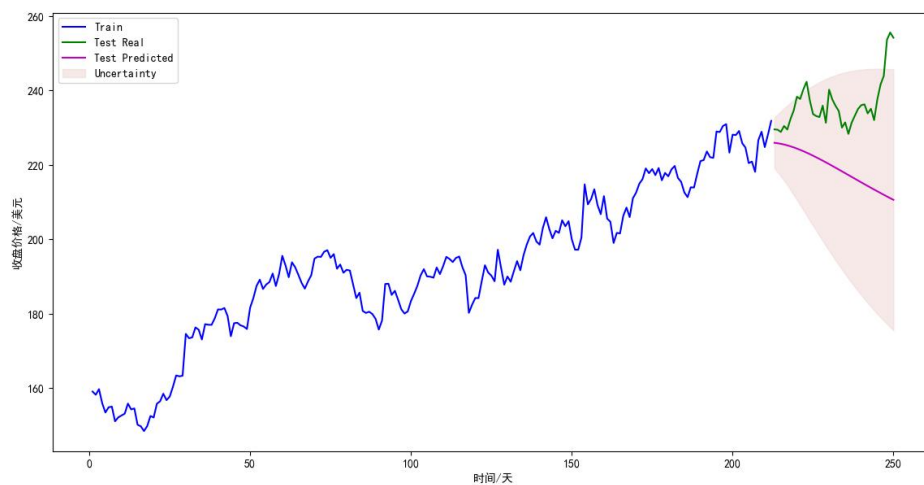
平均绝对百分比误差为：1.38%

实际值在预测上下限区间的概率为：93.03%

4.5 SAP 股票

4.5.1 Matérn 函数+ WhiteKernel

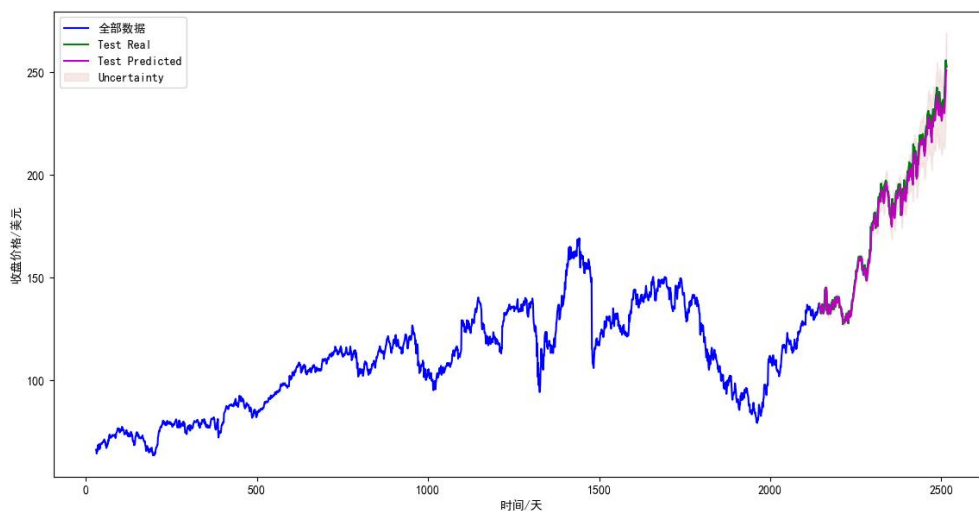
全局特征 GPR（1 年数据）：



平均绝对百分比误差为：7.12%

实际值在预测上下限区间的概率为：84.21%

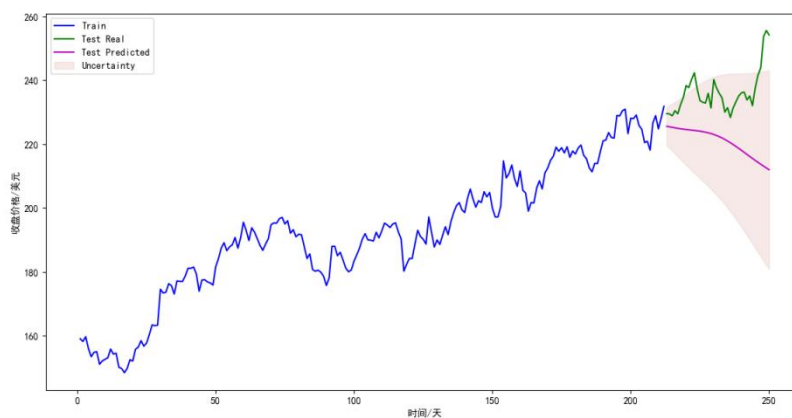
使用滑动窗口 GPR（10 年数据）：



平均绝对百分比误差为：1.40%
实际值在预测上下限区间的概率为：80.16%

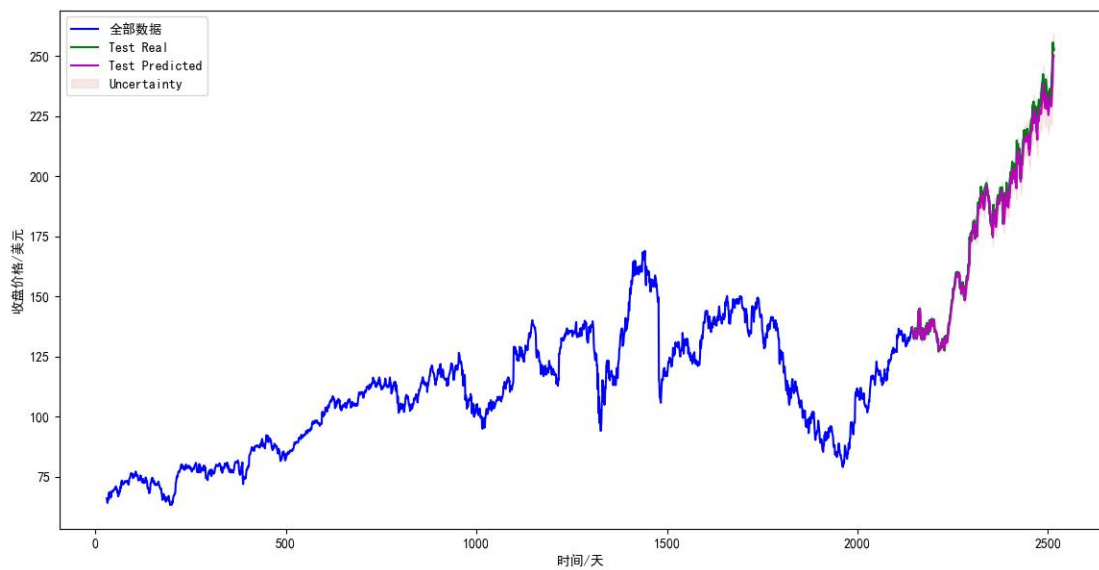
4.5.2 Rational Quadratic * Periodic + WhiteKernel 函数

全局特征 GPR（1 年数据）：



平均绝对百分比误差为：6.40%
实际值在预测上下限区间的概率为：78.95%

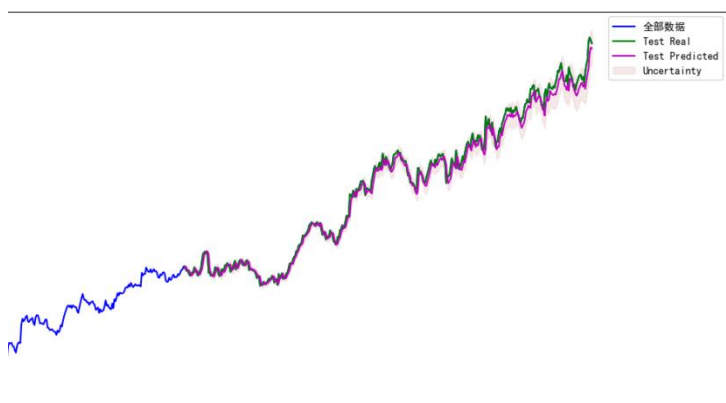
使用滑动窗口 GPR（10 年数据）：



平均绝对百分比误差为：1.42%

实际值在预测上下限区间的概率为：69.17%

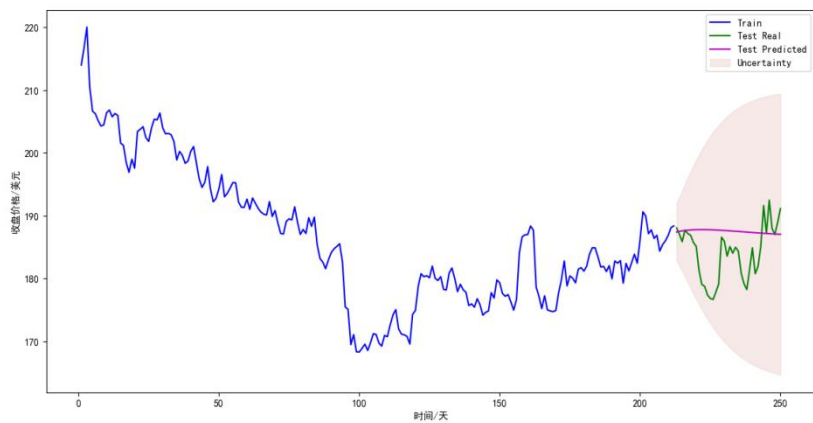
局部放大：



4.6 VRSN 股票

4.6.1 Matérn 函数+ WhiteKernel

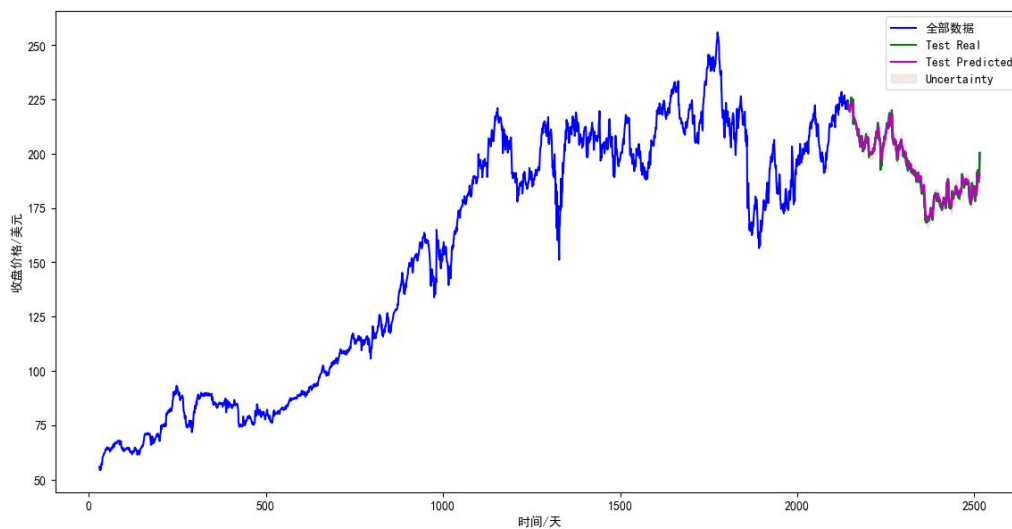
全局特征 GPR（1 年数据）：



平均绝对百分比误差为：2.41%

实际值在预测上下限区间的概率为：100.00%

使用滑动窗口 GPR（10 年数据）：

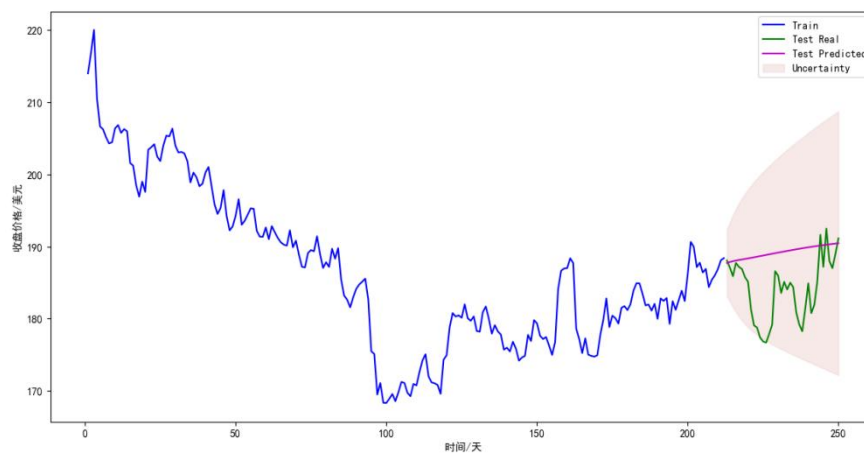


平均绝对百分比误差为：0.96%

实际值在预测上下限区间的概率为：79.09%

4.6.2 Rational Quadratic * Periodic + WhiteKernel 函数

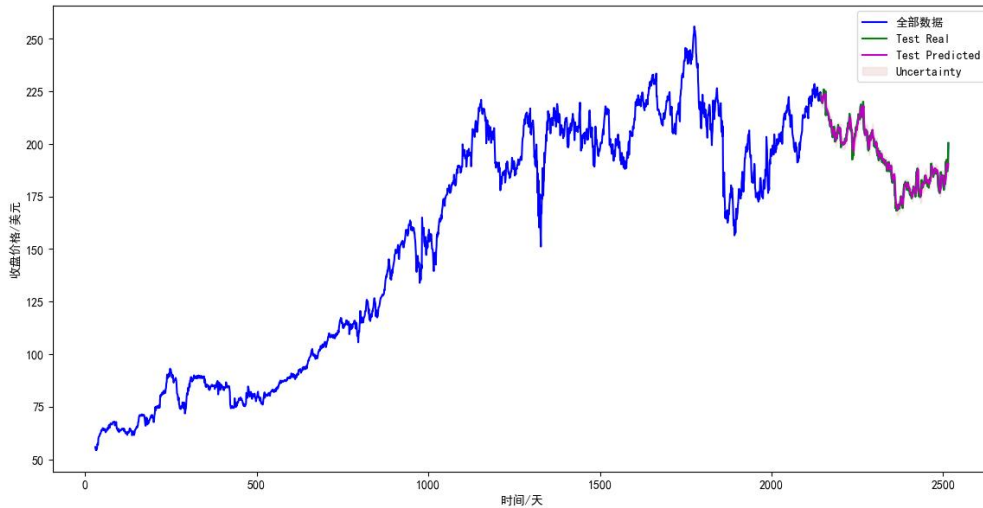
全局特征 GPR（1 年数据）：



平均绝对百分比误差为：2.99%

实际值在预测上下限区间的概率为：92.11%

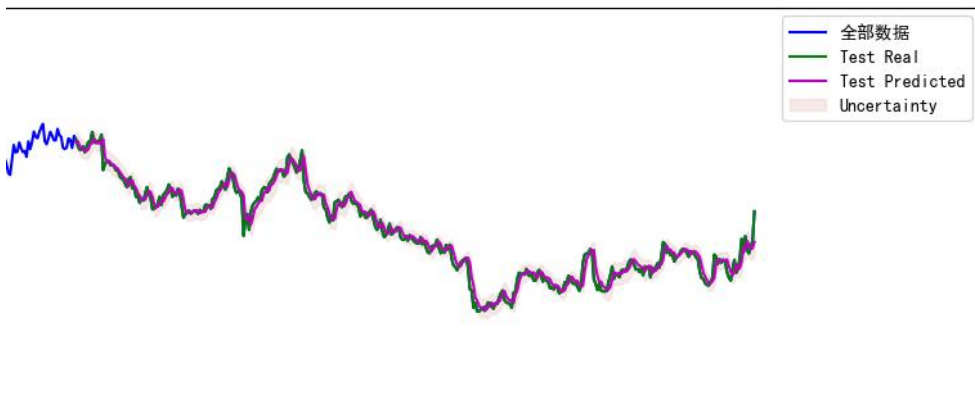
使用滑动窗口 GPR（10 年数据）：



平均绝对百分比误差为：0.95%

实际值在预测上下限区间的概率为：77.75%

预测部分局部放大如下：



4.6.3 对比分析

在对 VRSN 股票的预测中，使用 Matérn 函数与 WhiteKernel 组合的全局特征 GPR 模型在 1 年数据上的平均绝对百分比误差为 2.41%，并且所有实际值都落在预测上下限区间内，显示了极高的置信度。相比之下，采用相同核函数但基于滑动窗口方法的 GPR 模型在 10 年数据上的表现更为优异，其平均绝对百分比误差降低至 0.96%，尽管实际值落在预测区间的概率降至 79.09%。这表明滑动窗口方法能够更好地捕捉长期趋势，同时保持较低的预测误差。

另一方面，使用 Rational Quadratic 与 Periodic 核及 WhiteKernel 组合的 GPR 模型在 1 年数据上的平均绝对百分比误差略高，达到 2.99%，而实际值落在预测区间的概率为 92.11%。对于 10 年数据的滑动窗口 GPR 模型，该组合的平均绝对百分比误差进一步下降到 0.95%，但实际值落在预测区间的概率也有所减少，为 77.75%。这种组合虽然在捕捉周期性模式方面表现出色，但在长期内预测的不确定性较大，导致置信区间包含实际值的概率稍低。

总体来看，滑动窗口方法无论是在 Matérn 还是 Rational Quadratic * Periodic + WhiteKernel 核下，都能显著降低平均绝对百分比误差，显示出更好的短期预测能力。然而，随着预测时间范围的增加，即使滑动窗口方法能够提供更精确的点预测，预测区间的置信度却有所下降。这一现象可能归因于滑动窗口方法在处理较长时间序列时，对

远期预测的不确定性和波动性估计不足。此外，全局特征 GPR 模型在 1 年数据上的高置信度反映了它对近期数据的良好拟合，但其预测性能未能在 10 年的时间尺度上超越滑动窗口方法。

4.7 预测步长分析

前面的分析使用的都是 pred_steps 为 1,即使用 30 天的滑动窗口预测其后一天的收盘价。这个方法的周期有些过短。而对于不使用滑动窗口的全局特征方式则是直接用 1 年的数据预测后面 50 天的，周期又过长。因此引入了 pred_steps 参数来调整预测的步长，以使模型能够应用于不同时间段长度的预测，同时分析其效果。该部分分析主要作用是屏蔽滑动窗口的影响，直接地评估模型对较长时间段的预测效果

下面将 pred_steps 调整为了 5,即采用前面 30 天的数据,对后续五天的收盘价进行预测。对 ADSK 和 MSFT 两只股票进行了预测。

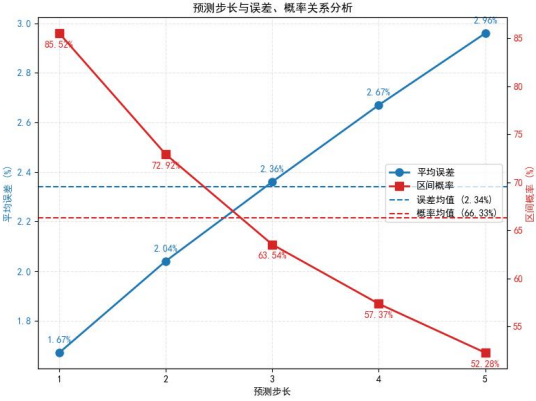
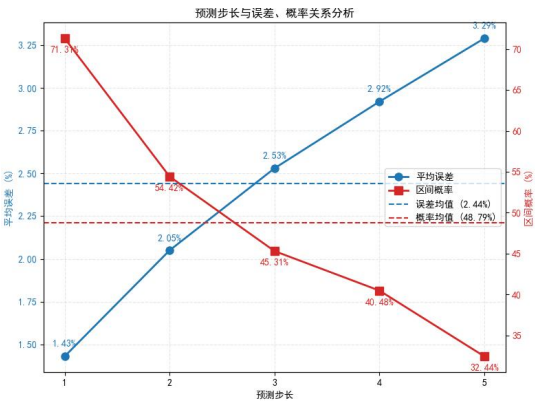
预测结果分析：		
步骤	平均误差	区间概率
1	1.43%	71.31%
2	2.05%	54.42%
3	2.53%	45.31%
4	2.92%	40.48%
5	3.29%	32.44%
总体	2.44%	48.79%

ADSK

预测结果分析：		
步骤	平均误差	区间概率
1	1.67%	85.52%
2	2.04%	72.92%
3	2.36%	63.54%
4	2.67%	57.37%
5	2.96%	52.28%
总体	2.34%	66.33%

MSFT

可以看到随步长的增加，平均误差急剧增大，同时预测值位于预测上下限区间内的概率也同步的下降。对于第 5 天的预测，其误差达到了第一天预测误差的两倍左右，位于区间内的概率则下降到了二分之一附近。再对预测的平均误差与区间概率绘图进行分析，可以看到误差的增大与区间概率的降低几乎是呈线性的：



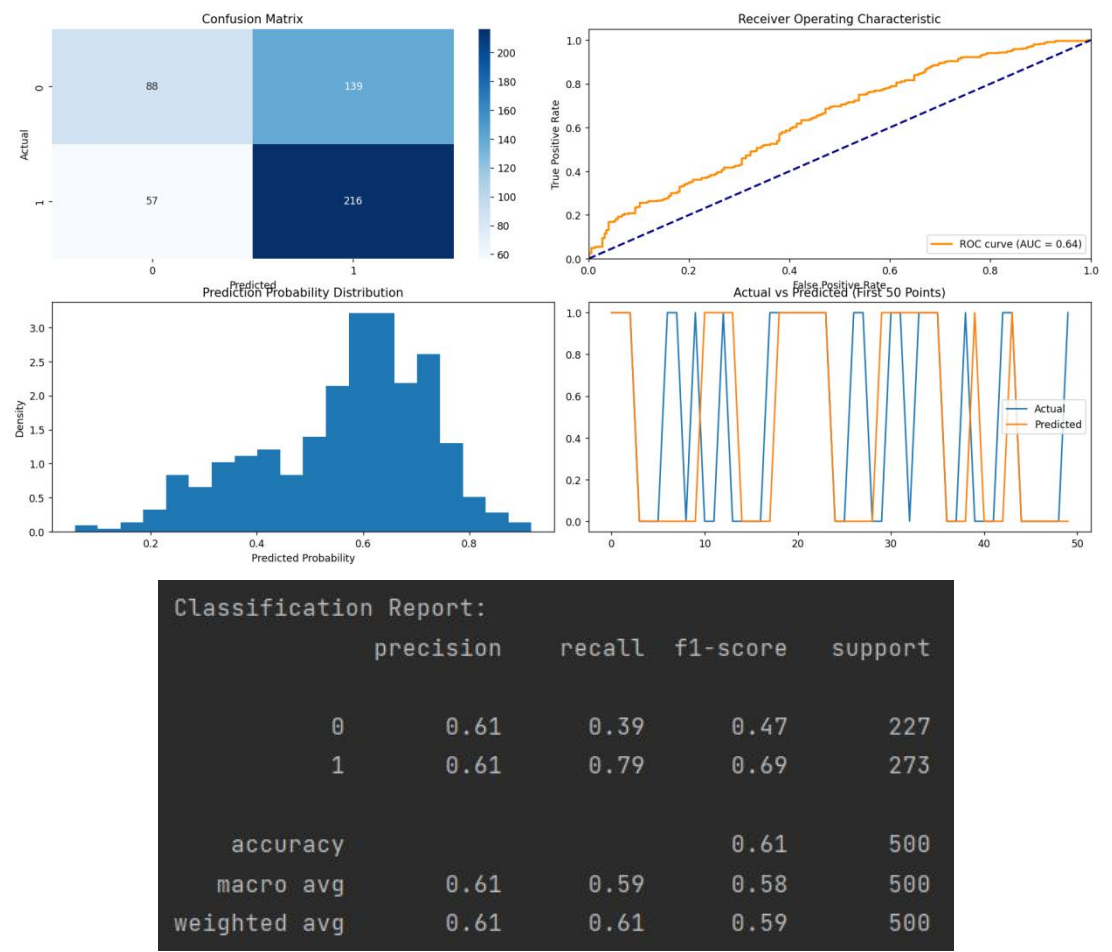
因此随着预测步长的增加，预测准确性下降非常的严重，会极大地影响到模型的性能。这也侧面反映出了滑动窗口的有效性，能够弥补对于长期预测的性能不足。

4.8 涨跌预测分析

在本节中，我们基于上述过程中创建的核函数以及滑动窗口的应用，通过滚动预测，将预测值反标准化后转换为涨跌预测（0/1），并通过与实际涨跌情形进行效果分析对比，来具体分析已创建模型对涨跌预测的准确率。如下所示，我们选取了三只股票 MSFT、VRSN、OCRL 分别进行了高斯分类与高斯回归后判断涨跌的效果分析：

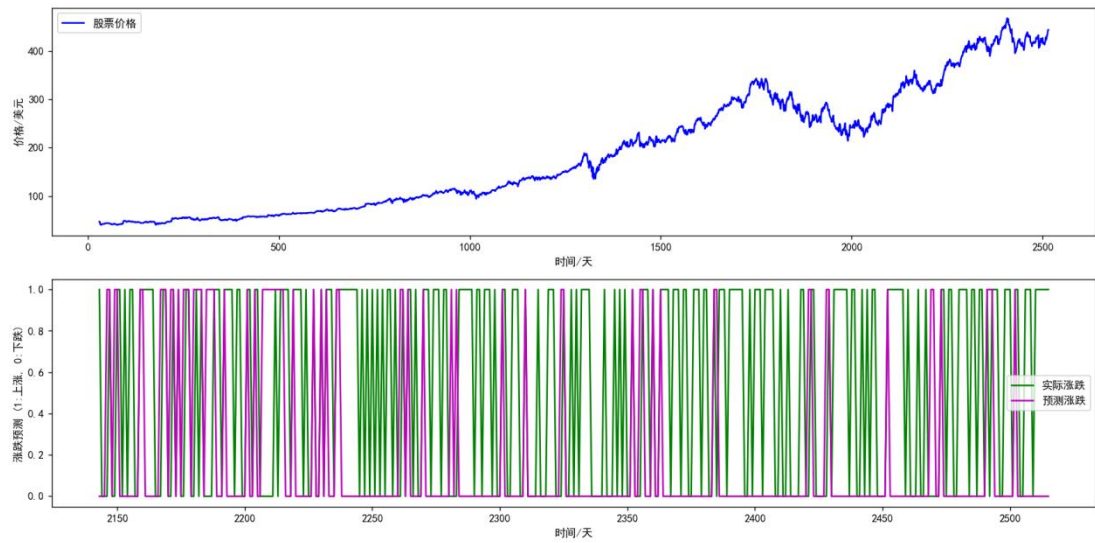
4.8.1 MSFT 股票

我们进行了如下的分类效果分析，通过混淆矩阵的结果，我们可以发现对该股票来说，整体模型可能存在对正类的偏向性（倾向于预测为 1），说明模型在负类样本上的区分能力较弱。



接下来我们对两类核函数组合分别进行了高斯回归的涨跌预测

Rational Quadratic * Periodic + WhiteKernel:

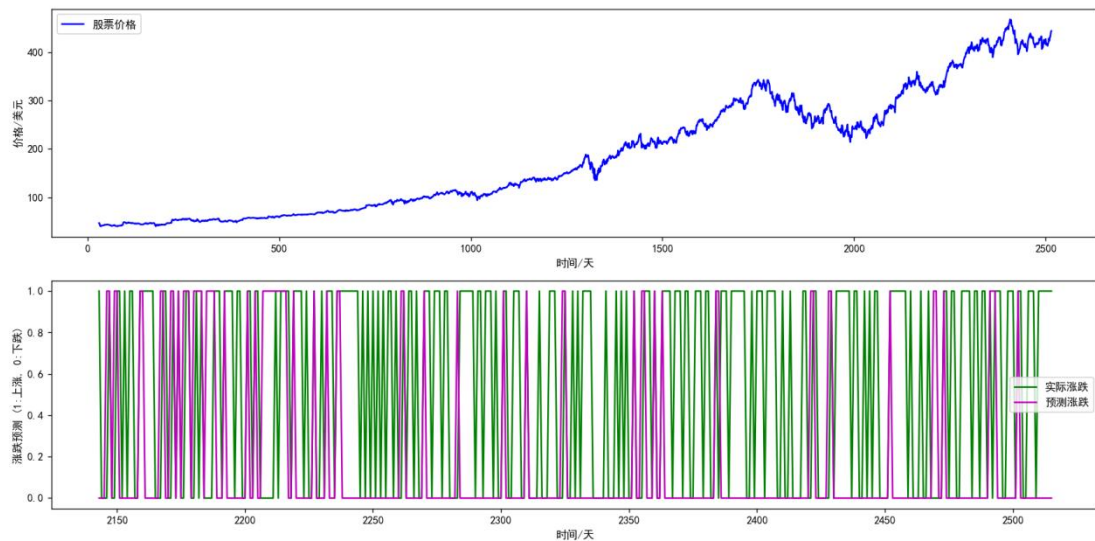


涨跌预测准确率: 52.01%

涨跌预测分类报告:

	precision	recall	f1-score	support
0	0.47	0.70	0.57	166
1	0.61	0.37	0.46	207
accuracy			0.52	373
macro avg	0.54	0.54	0.51	373
weighted avg	0.55	0.52	0.51	373

Matern +WhiteKernel:



涨跌预测准确率：48.53%

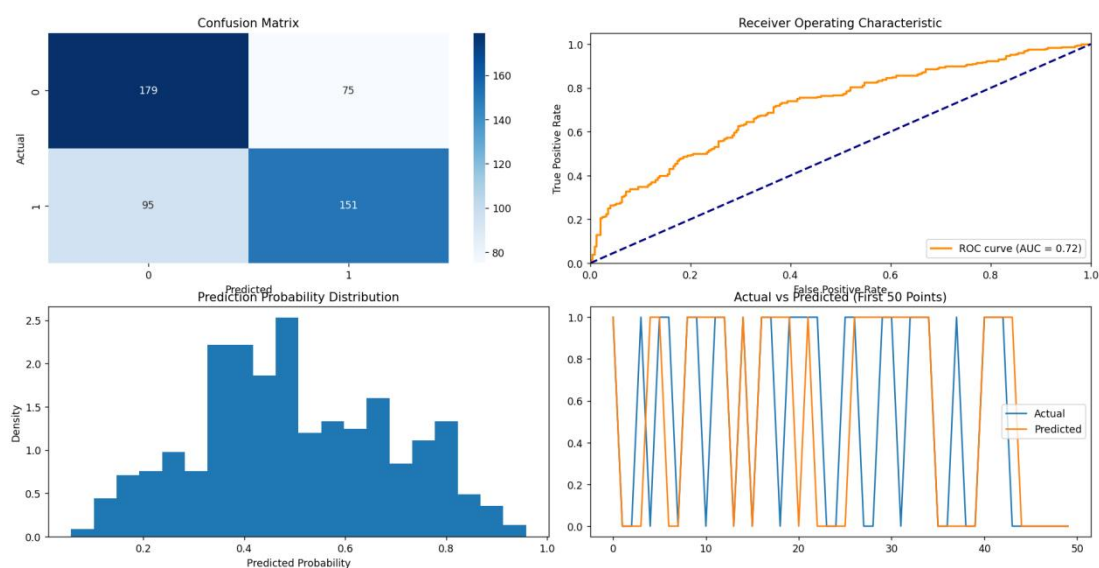
涨跌预测分类报告：

	precision	recall	f1-score	support
0	0.46	0.84	0.59	166
1	0.61	0.20	0.30	207
accuracy			0.49	373
macro avg	0.53	0.52	0.45	373
weighted avg	0.54	0.49	0.43	373

通过对比分析，我们可以发现两种组合对于涨跌预测准确率的效果近似，而通过对比性能分析，我们分析模型对于涨跌预测的波动性较差，这可能是由于真实数据集的动态变化较大。

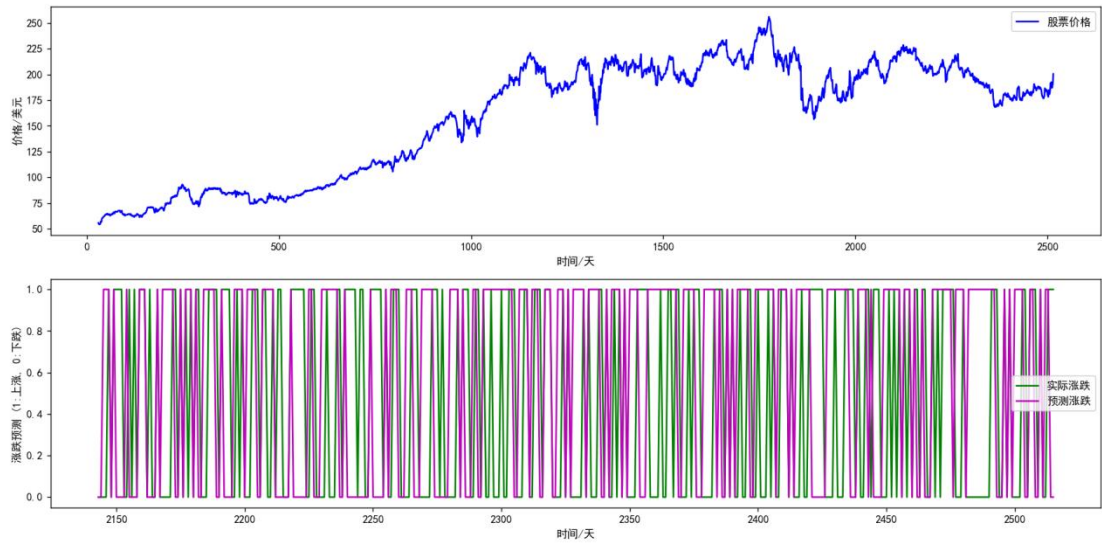
通过对比分类与回归的效果，我们可以发现直接分类的准确率更高，这间接说明了高斯回归对于连续单日的涨跌预测并不是很精确，而且其更偏向于涨的预测，结合先前小样本滑动窗口的股票价格预测，我们发现回归后预测的大致趋势与价格走向一致，因此涨跌预测并不决定性影响股票价格预测走向。

4.8.2 VRSN 股票



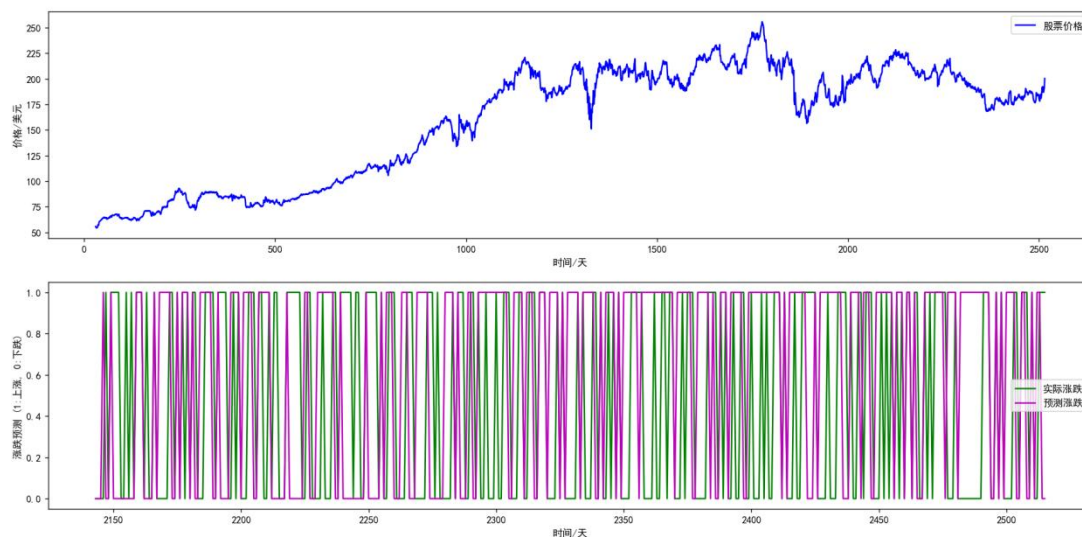
Classification Report:				
	precision	recall	f1-score	support
0	0.65	0.70	0.68	254
1	0.67	0.61	0.64	246
accuracy			0.66	500
macro avg	0.66	0.66	0.66	500
weighted avg	0.66	0.66	0.66	500

Rational Quadratic * Periodic + WhiteKernel:



涨跌预测准确率：49.60%				
涨跌预测分类报告：				
	precision	recall	f1-score	support
0	0.52	0.40	0.45	193
1	0.48	0.60	0.53	180
accuracy			0.50	373
macro avg	0.50	0.50	0.49	373
weighted avg	0.50	0.50	0.49	373

Matern +WhiteKernel:

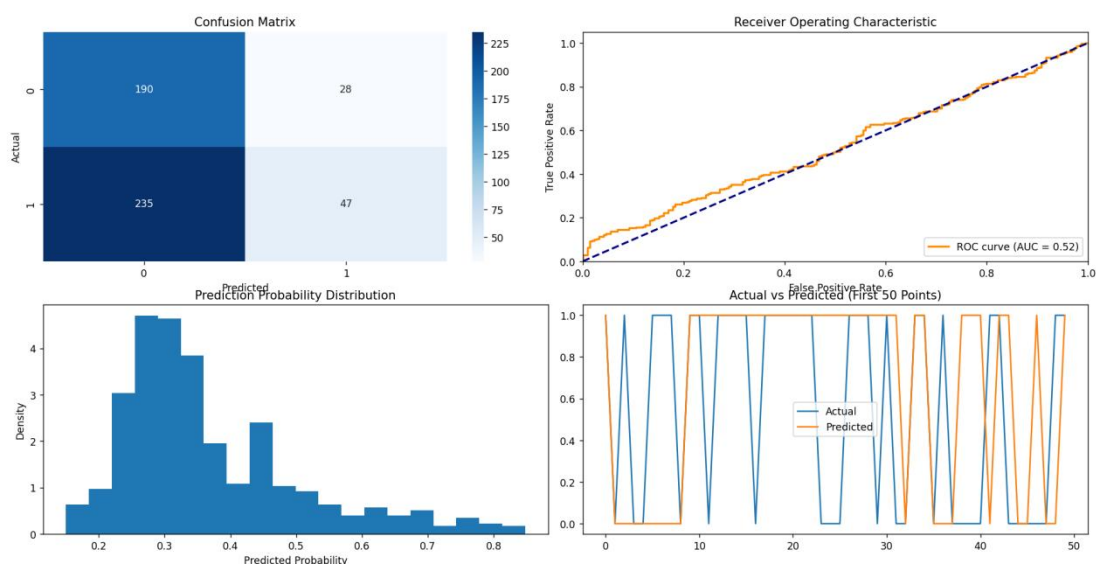


涨跌预测准确率: 51.74%

涨跌预测分类报告:

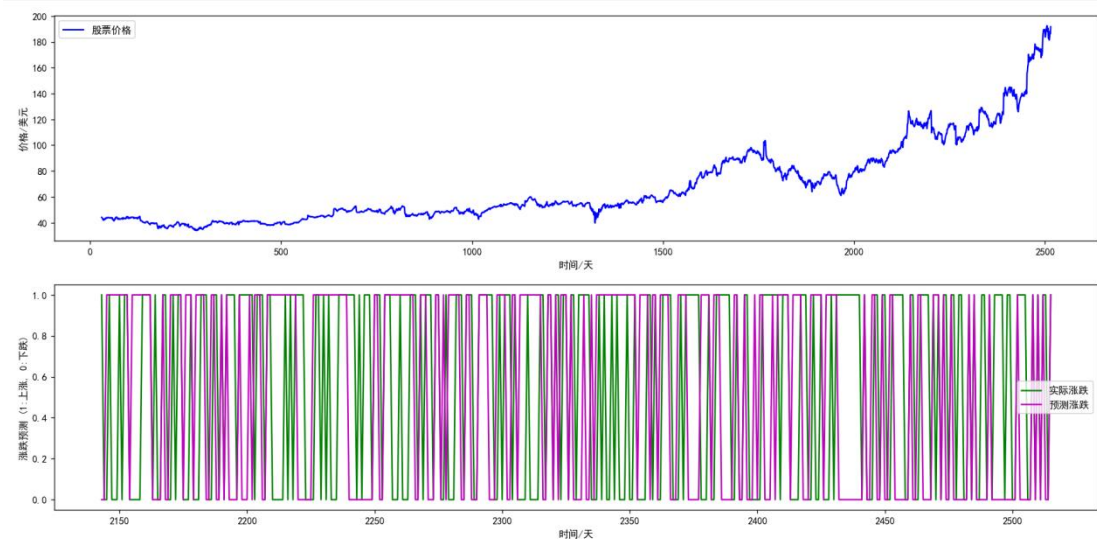
	precision	recall	f1-score	support
0	0.55	0.39	0.46	193
1	0.50	0.65	0.57	180
accuracy			0.52	373
macro avg	0.52	0.52	0.51	373
weighted avg	0.52	0.52	0.51	373

4.8.3 OCRL 股票



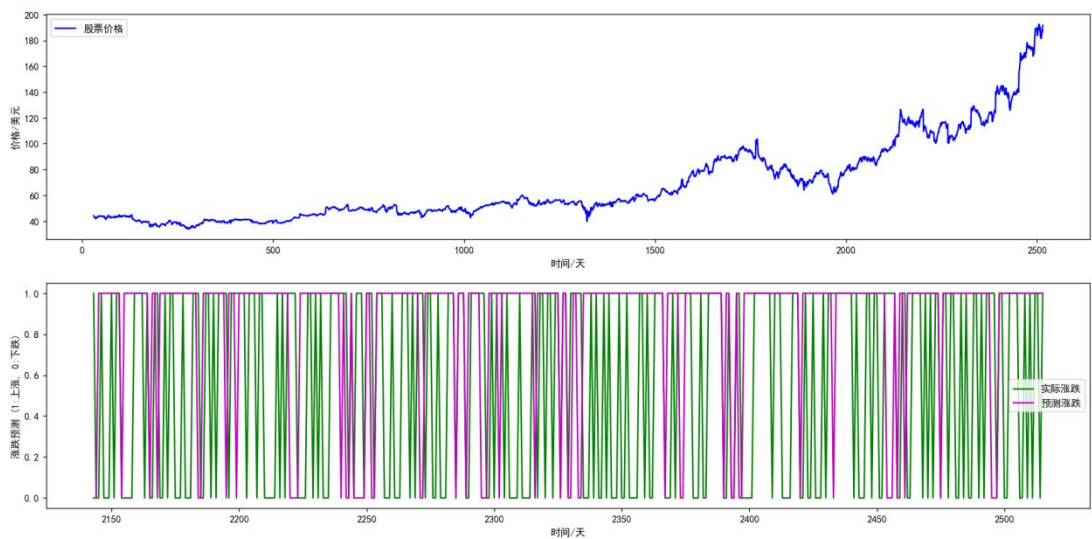
Classification Report:				
	precision	recall	f1-score	support
0	0.45	0.87	0.59	218
1	0.63	0.17	0.26	282
accuracy			0.47	500
macro avg	0.54	0.52	0.43	500
weighted avg	0.55	0.47	0.41	500

Rational Quadratic * Periodic + WhiteKernel:



涨跌预测准确率: 50.40%				
涨跌预测分类报告:				
	precision	recall	f1-score	support
0	0.44	0.45	0.44	165
1	0.56	0.55	0.55	208
accuracy			0.50	373
macro avg	0.50	0.50	0.50	373
weighted avg	0.50	0.50	0.50	373

Matern + WhiteKernel:



涨跌预测准确率：53.08%

涨跌预测分类报告：

	precision	recall	f1-score	support
0	0.41	0.14	0.21	165
1	0.55	0.84	0.67	208
accuracy			0.53	373
macro avg	0.48	0.49	0.44	373
weighted avg	0.49	0.53	0.46	373

4.8.4 对比分析

对于 MSFT 股票，整体模型存在对正类的偏向性，在负类样本区分能力较弱，Rational Quadratic * Periodic + WhiteKernel 组合与 Matern + WhiteKernel 组合的涨跌预测准确率分别为 52.01% 和 48.53%，效果近似，且模型预测波动性较差。VRSN 股票整体准确率达 66%，不同核函数组合准确率相近，Rational Quadratic * Periodic + WhiteKernel 组合为 49.60%，Matern + WhiteKernel 组合为 51.74%，整体性能较为接近。OCRL 股票整体准确率为 47%，模型性能一般，Rational Quadratic * Periodic + WhiteKernel 组合准确率为 50.40%，Matern + WhiteKernel 组合为 53.08%。

对于分类和回归预测这两种方式而言，直接分类的效果可能稍优于回归预测的效果，这可能是因为回归是具体计算量连续变量的走向，并给出了频率更高的总体走向，而分类由于是对已有样本的离散预测，故可能对单日的预测结果表现得更加良好。

综合来看，三只股票在不同核函数组合下涨跌预测准确率多在 50% 左右波动，部分股票存在模型偏向性，预测波动性较大，不同核函数组合在不同股票上表现有差异，无绝对优势组合，表明核函数组合选择需结合股票数据特性优化。

对比预测与实际的折线图，我们可以发现模型对于涨跌的预测走向与实际走向存在较大偏差，但结合前面章节基于滑动窗口的预测效果，整体股票价格预测值的平均绝对百分比误

差已经极低，结合分析，其实涨跌的预测走向对预测价格没有很大的影响，可能涨/跌都只是进行了小幅度的变化，因此预测价格处于良好的置信区间内。

4.9 时间复杂度分析

GPR 的时间复杂度主要由两个因素决定：核函数的计算和求解协方差矩阵。对于一个包含 n 个训练样本的数据集，构建协方差矩阵需要 $O(n^2)$ 的操作，而求解该矩阵（例如计算其逆或进行 Cholesky 分解）通常需要 $O(n^3)$ 的时间复杂度。因此，当数据集较大时，GPR 的计算成本会迅速增加，使得它在处理大规模数据时不太实用，实验中也发现在 262 条的小样本数据上模型运行平均只需要 3 秒，而换成 2516 条的大样本数据，模型运行时间大幅增加至 5 分钟左右。

在本次实验中，在从指定个随机起点开始寻找核函数超参数的最佳值时，使用了 L-BFGS-B 优化算法进行模型超参数的估计，这是一种拟牛顿法，通过构建 Hessian 矩阵的近似来避免直接计算和求逆 Hessian 矩阵，从而减少每次迭代的计算成本，提高计算效率。

五.实验总结

本次实验应用高斯过程回归（GPR）模型对 ADBE、ADSK、MSFT、ORCL、SAP 和 VRSN 等股票的历史收盘价进行了预测。实验结果显示，对于大数据集（10 年数据），滑动窗口方法显著优于全局特征方法，能够更好地捕捉时间序列中的短期动态和局部模式，提供更高的预测精度和合理的置信度。然而，对于小数据集（1 年数据），全局特征方法表现通常更好，能够捕捉整体趋势，避免过拟合并提供更高的置信度。以 ADBE 为例，不使用滑动窗口的 GPR 模型在 1 年数据集上的 MAPE 为 4.07%，且 100% 的实际值落在预测上下限区间内，表明模型对不确定性的估计非常保守。相比之下，滑动窗口方法虽然 MAPE 略有下降至 1.42%，但置信度显著下降至 60.61%，这可能是因为滑动窗口方法在小数据集上稳定性较差，容易受到噪声和异常值的影响。

进一步分析不同核函数组合的结果发现，Matérn + WhiteKernel 组合在小数据集上略优于 Rational Quadratic * Periodic + WhiteKernel 组合，可能是因为前者对噪声的鲁棒性更强，更适合处理有限的数据。而在大数据集上，两种核函数组合都表现出色，尤其是 Rational Quadratic * Periodic + WhiteKernel 组合在处理具有复杂动态变化的时间序列时，能够捕捉多尺度的相关性和周期性，提供了较高的预测精度和合理的置信度。

综上，滑动窗口方法在大数据集上显著提高了预测精度和置信度，而全局特征方法在小数据集上更为稳定可靠，这一结论验证了 GPR 模型在股票价格预测中的应用效果。实验结果表明，选择合适的模型结构和参数对于提升预测性能至关重要，同时也强调了在实际应用中需要根据数据特性和预测目标灵活调整模型的重要性。此外，实验还展示了高斯过程回归作为一种非参数方法的优势，它能够灵活处理非线性关系并提供预测的不确定性估计，这对于投资者进行风险管理和决策制定具有重要意义。

最后，我们还对涨跌的预测进行了分类与回归的对比分析，回归的预测偏向可能直观反映了整体的股票价格走向，但实际上，整体的股票预测价格走向与准确的涨跌预测不存在决定性影响，涨/跌在连续日期内都只是进行了小幅度的变化，因此预测价格处于良好的置信区间内。

通过本次大作业，我充分了解并实践了高斯过程回归的方法在股票价格预测方面的应用。预测结果表明，高斯过程回归的方法对纳斯达克股票市场的 6 支股票都呈现出可接受的预测结果，能够有效地应用于股票价格预测，论证了使用高斯过程回归模型预测股票价格的可行性。该模型计算方便，能给投资者提供长期或短期的预测与决策。但本实验也有局限性，实际上，使用高斯过程并不是最有效的方法，因为预测的计算复杂度很高。要使用机器学习技术在股票市场赚钱，需要更快的计算方法。

参考文献

- [1] C.K. I. Williams, C.E. Rasmussen “Gaussian Processes for Regression”
- [2] M.Ebden, " Gaussian Processes for Regression An Quick Introduction".
- [3] H.M.Wallace, “Introduction to Gaussian Process Regression”
- [4] Z. Ghahramani, ” A Tutorial on Gaussian Processes”
- [5] M.T. Farrell, et al, “Gaussian Process Regression Models for Predicting Stock Trends” .
- [6] Long-term Stock Market Forecasting using Gaussian Processes
- [7] J.M, Tomczak, et, al, “Gaussian process regression as a predictive model for Quality-of-Service in Web service systems”
- [8] Y. Altmann, et,al, “Nonlinear spectral unmixing of hyperspectral images using Gaussian processes” .
- [9] H. Topa, et,al, “Gaussian process modelling of multiple short time series” .
- [10] Chuong B. Do, ” Gaussian Processes”
- [11] C.J. Paciorek, ” NONSTATIONARY GAUSSIAN PROCESSES FOR REGRESSION AND SPATIAL MODELING ”